

VPJoin: Verifiable and Privacy-Preserving Multi-Way Join via Zero-Knowledge Proofs

Binbin Gu
Nanyang Technological University
Singapore

Yihua Hu
Nanyang Technological University
Singapore

Wei Dong
Nanyang Technological University
Singapore

Abstract

Modern applications increasingly demand both data confidentiality and query integrity in SQL query answering. Zero-knowledge (ZK) proofs address this by allowing a prover to demonstrate the correct execution of SQL queries on private data without revealing the data itself. However, efficiently proving multi-way joins poses a fundamental challenge rooted in a seemingly irreconcilable tension: existing solutions need to materialize all intermediate results and use them to perform the verification. Since intermediate join results are data-dependent and thus privacy-sensitive, such solutions must further pad every intermediate to the worst-case size. As a result, for the join query involving k relations, their verification requires executing a circuit with the size of $O(IN^k)$, where IN is the total input size, rendering even modest multi-way joins (e.g., $k \geq 3$) computationally infeasible.

To address this issue, we present **VPJoin**, which resolves this tension through a paradigm shift from *proving-by-re-execution* to *proving-by-correctness-verification*: rather than materializing all intermediate join results, the prover supplies a more compact witness, while the verification is done through lightweight structural checks. For acyclic joins, we introduce the **Oblivious Join Gate (OBJ)**, which verifies semijoin-based reductions along a join tree and achieves $O(IN + OUT)$ complexity, where OUT is the join result size. OBJ also supports join-aggregate queries with the same cost, where OUT denotes the final aggregated output, by propagating tuple multiplicities without materializing the full join. For cyclic joins, we develop the **Tree-Decomposed Join Gate (TDJ)**, which confines materialization to small clusters via tree decomposition while remaining fully oblivious; optionally, applying DP-guided padding on top further reduces the intra-cluster padding cost in exchange for a quantified (ϵ, δ) leakage of intra-cluster capacities. Experiments on TPC-H and graph-pattern benchmarks show that VPJoin outperforms the state-of-the-art by 5–18 orders of magnitude in proving time.

Keywords

zero-knowledge proofs, verifiable databases, multi-way joins

1 Introduction

Organizations increasingly publish analytical results derived from sensitive databases, including regulatory filings, public-health statistics, and cross-organization analytics. A fundamental question arises: how can a data holder convince external parties that a reported query result is correct, computed faithfully over the committed dataset, without revealing the underlying private records? Valid solutions to this problem require two properties: *verifiability* (external parties can confirm the result was computed correctly) and *privacy* (the underlying data records remain hidden during the verification). For example, a government agency may publish unemployment statistics computed from a private census database: external auditors need assurance that

the reported figures are accurate, yet must not gain access to any individual record.

To achieve these goals, zero-knowledge (ZK) proofs [20, 22, 24, 30, 33, 37, 43] were introduced. In this setting, there are two parties: a *prover*, who holds a private database D and claims a query Q together with its result $Q(D)$, and a *verifier*, who seeks to be convinced that the claim is correct without learning the underlying data. At a high level, the prover first publishes a cryptographic *commitment* to D , which binds all subsequent proofs to the same database instance. To generate a proof, the prover constructs a private *witness* consisting of the input database and auxiliary values needed to certify the computation. The relationships between this witness and the claimed result are then encoded as an arithmetic *circuit*. Using the circuit and the witness, the prover generates a proof π , and the verifier checks π to confirm that $Q(D)$ is indeed the correct output of Q on the committed database D , thereby ensuring verifiability. At the same time, the cryptographic guarantees of zero knowledge ensure that π reveals nothing about D beyond the query result $Q(D)$ itself, thereby preserving privacy. To maintain this guarantee, neither the circuit structure nor the witness size, which directly affects the circuit, should leak private information about the data. In terms of efficiency, verification can often be performed in logarithmic time, while the main bottleneck lies in proof generation, whose cost is roughly linear in the circuit size (see Section 3.2 for more details). Henceforth, we use *circuit complexity* to refer to the cost of proof generation, since the two are deterministically related and nearly linear.

Applying ZK proofs to query answering over relational databases has been widely studied [26, 32, 53, 54]. Recent ZK database systems such as ZKSQL [32] and PonglyphDB [26] have demonstrated that ZK proofs for single-table operations such as sort, range check, and binary joins are efficient and practical. However, these works do not efficiently support multi-way joins, despite the fact that multi-way joins are fundamental to relational analytics and ubiquitous in practice. To date, all existing ZK database systems [26, 32] decompose a k -way join into $k-1$ successive binary joins and supply every intermediate join result as part of the witness to the verification circuit. To prevent privacy leakage through the witness size, which is publicly revealed, these intermediate join results must be padded to their worst-case sizes. This padding compounds multiplicatively across the $k-1$ binary join steps, inflating the circuit size to $O(IN^k)$, where IN denotes the total size of the input database. As discussed above, the proving time scales with circuit size, thus resulting in prohibitive costs in practice. For example, for TPC-H Q3, which involves a 3-way join, PonglyphDB [26] estimates a proving time of more than 30 years; for Q8, which involves an 8-way join, the estimated proving time exceeds 10^{13} years, far beyond the age of the universe (these numbers are *extrapolated estimates* derived from circuit sizes without

the schema constraints; Section 8 details the estimation methodology and its assumptions.). This naturally leads to a fundamental question:

“Can multi-way joins be verified in zero knowledge without materializing all intermediate results, thereby avoiding the worst-case circuit blowup?”

1.1 Our Results

In this paper, we answer this question affirmatively by presenting **VPJoin**, which supports both acyclic and cyclic multi-way join queries based on a simple yet powerful insight. As discussed earlier, designing a ZK-proof implementation ultimately reduces to constructing an efficient circuit that captures the relationship between the input and the output. For multi-way joins, however, this relationship can be expressed much more efficiently than by explicitly re-executing every binary join step, as existing systems do, which forces all intermediate join results into the witness in the first place. This paradigm shift, from *proving-by-re-execution* (encoding each binary-join step inside the circuit) to *proving-by-correctness-checking* (checking only that the prover’s supplied result is correct), eliminates intermediate materialization entirely.

Building on this insight, we first design the **Oblivious Join Gate (OBJ)**, a circuit to verify acyclic joins, i.e., joins whose join graph admits a tree structure, inspired by classical semijoin reductions along a join tree [9, 48]. The high-level idea is that the prover splits each input table into two groups, *clean* tuples (those that participate in the join result) and *residual* tuples (those that do not), and the circuit verifies that this split is correct through several lightweight structural checks. Each check scans the input tables and output join results at most a constant number of times, yielding a total circuit complexity linear in the input and output sizes, i.e., $O(IN + OUT)$. Furthermore, we extend OBJ to support projection, predicates, and aggregation on the join result, all without materializing the full intermediate join results. The circuit complexity remains $O(IN + OUT)$, where OUT denotes the size of the final aggregated output rather than the full join result. For cyclic joins, our proposed OBJ does not work since it relies on the existence of a join tree, which requires the query to be acyclic. To address this, we propose the **Tree-Decomposed Join Gate (TDJ)**, which reduces a cyclic query to an acyclic one via *tree decomposition* [23, 28, 39]: the relations are grouped into a small number of *clusters* such that, at the inter-cluster level, each cluster acts as a single “super-relation” and the clusters together form a join tree whose structure is acyclic, so OBJ applies directly. This confines the intermediate blowup from $O(IN^k)$ for the original query to $O(IN^w)$ within each cluster, where w is the treewidth [23] (typically $w \leq k$). However, these intra-cluster intermediates are not true input relations but data-dependent materialized results, so their sizes must be hidden to preserve privacy. By padding each intra-cluster intermediate to its worst-case bound, TDJ hides these sizes completely, so cyclic queries enjoy the same *full obliviousness* guarantee as acyclic ones. TDJ itself uses no differential privacy. When full obliviousness is not required, one can further apply *differential privacy* [15] on top of TDJ, setting each intra-cluster capacity from a DP-guided bound instead of its worst-case bound. In summary, this paper makes the following contributions:

- For acyclic join queries, we propose OBJ, which supports projection, predicates, and aggregation, all without materializing intermediate results. Compared with prior work whose circuit size

grows to $O(IN^k + OUT)$, OBJ achieves $O(IN + OUT)$ circuit complexity, where OUT is the size of the final output. For aggregation queries, OUT is the final aggregated output size rather than the full join result. Notably, for aggregation queries, this means our result also improves binary join-aggregate queries, from a circuit complexity of $O(IN^2 + OUT)$ to $O(IN + OUT)$.

- For cyclic joins, we propose TDJ, which extends OBJ to support cyclic join verification. First, through tree decomposition, we achieve a circuit complexity of $O(IN^w + OUT)$, where w is the treewidth, usually much smaller than k , while remaining fully oblivious via worst-case intra-cluster padding. To further reduce the overhead of IN^w caused by padding part of intermediate results, we additionally allow DP-guided padding to be applied on top, achieving a better privacy–efficiency tradeoff.
- We conducted comprehensive experiments on five TPC-H and four graph queries. Experimental evaluation shows that VPJoin outperforms PoneglyphDB [26] by 8–18 orders of magnitude on TPC-H and 5–15 on graph workloads, reducing Q8 proving time from 10^{13} years to under 200 s.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 introduces preliminaries. Section 4 describes the problem setting. Section 5 presents the ZK proof primitives used throughout. Section 6 details the OBJ gate for acyclic joins. Section 7 extends to cyclic joins via TDJ, a fully oblivious construction with an optional DP-guided relaxation. Section 8 reports experimental results.

2 Related Work

Verifiable query processing ensures a result is correctly computed over a committed database; the zero-knowledge (ZK) subset additionally hides the underlying data from the verifier. Purely verifiable (non-ZK) systems based on authenticated data structures [5, 17, 36, 42, 47, 56] and query-authentication systems [7, 41, 52, 55] provide integrity but expose the underlying data. Our work is in the ZK subset, where the prover additionally hides the data. In this setting, vSQL [53, 54] compiles relational operators into circuits verified via the GKR protocol [21]. ZKSQL [32] decomposes verification into operator-level gadgets, improving modularity but requiring multiple rounds of interaction and coupling the circuit to a fixed query plan. PoneglyphDB [26] builds a practical non-interactive system on Halo2 [50, 51] with PLONKish arithmetization [19], supporting selections, projections, aggregations, sorting, and equi-joins. All of these systems decompose multi-way joins into binary joins, materializing every intermediate inside the circuit. Because circuit sizes are public, intermediate sizes leak cardinality information; padding them to worst-case sizes leads to an $O(IN^k)$ blowup. VPJoin eliminates this problem: its witness-guided verification never materializes intermediates, achieving $O(IN + OUT)$ for acyclic queries, and confining materialization to small clusters for cyclic ones, which remain fully oblivious by default.

Beyond the ZK setting, Secure Yannakakis [46] and SPECIAL [45] use the semijoin reduction strategy in multi-party computation (MPC), but rely on heavy cross-party primitives (PSI [34, 38], garbled circuits [49]) that do not translate efficiently into ZK circuits. Our single-prover model differs fundamentally: it produces publicly verifiable proofs with circuit size as the dominant cost.

3 Preliminaries

3.1 Notations

A database $D = \{R_1, R_2, \dots, R_k\}$ is a finite collection of k relations over a universe of attributes $\mathcal{V} = \bigcup_{i=1}^k \text{attr}(R_i)$, where $\text{attr}(R_i)$ is the schema of R_i and each attribute $A \in \mathcal{V}$ draws values from a finite domain $\text{Dom}(A)$. A tuple over an attribute set $F \subseteq \mathcal{V}$ is an element in $\text{Dom}^F := \prod_{A \in F} \text{Dom}(A)$, and each relation R_i is a finite multiset of tuples over $\text{attr}(R_i)$. For $t \in R_i$ and $F \subseteq \text{attr}(R_i)$, we write $t[F]$ for the restriction of t to F . The cardinality of R_i is denoted $|R_i|$, and the total input size is $IN = \sum_{i=1}^k |R_i|$. Following prior work [26, 32], we assume each $|R_i|$ (and hence $IN = \sum_i |R_i|$) is public.¹

We associate each query with a join hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, whose vertices are the attributes in $\mathcal{V}$ and whose hyperedges $F_i := \text{attr}(R_i) \in \mathcal{E}$ are the schemas of the input relations. A join tree of $\mathcal{H}$ is a separate tree T (not equal to $\mathcal{H}$) whose nodes are in one-to-one correspondence with the input relations $\{R_1, \dots, R_k\}$ (equivalently, with the hyperedges of $\mathcal{H}$) and which satisfies the *running-intersection property* [9]: for every attribute $A \in \mathcal{V}$, the nodes whose relations contain A induce a connected subtree of T . A query is *acyclic* if its hypergraph admits a join tree, and *cyclic* otherwise. We fix an arbitrary root of T and denote each tree edge as a parent-child pair (P, C) with shared attributes $K_{PC} := \text{attr}(P) \cap \text{attr}(C)$.

The projection of a relation R onto an attribute set $F \subseteq \text{attr}(R)$ is

$$\pi_F(R) = \{t[F] \mid t \in R\},$$

i.e., the set of distinct restrictions of R 's tuples to F . In addition, we use $R \equiv S$ to denote *multiset equality*: R and S contain the same tuples with the same multiplicities, which is strictly stronger than set equality.

The *natural join* of $R_1, \dots, R_k$ is the set of tuples over $\mathcal{V}$ whose projection onto each schema $\text{attr}(R_i)$ is a tuple of R_i :

$$J(D) := R_1 \bowtie R_2 \bowtie \dots \bowtie R_k = \{t \in \text{Dom}^{\mathcal{V}} \mid \pi_{\text{attr}(R_i)}(t) \in R_i \text{ for all } i \in [k]\}.$$

Equivalently, two tuples $t \in R_i$ and $s \in R_j$ combine when they agree on their shared attributes $K_{ij} := \text{attr}(R_i) \cap \text{attr}(R_j)$, i.e., $t[K_{ij}] = s[K_{ij}]$.

The *semijoin* and *antijoin*, used to isolate matching and non-matching tuples along a single join edge, are defined for any $i \neq j$ as

$$R_i \ltimes R_j = \{t \in R_i \mid \exists s \in R_j : t[K_{ij}] = s[K_{ij}]\},$$

$$R_i \triangleright R_j = \{t \in R_i \mid \forall s \in R_j : t[K_{ij}] \neq s[K_{ij}]\}.$$

Finally, we consider *aggregation* on top of a join, as is common in analytical SQL queries. For a grouping attribute set $G \subseteq \mathcal{V}$ and an aggregation attribute $A \in \mathcal{V}$ (or the wildcard $A = *$, which represents “no attribute” and yields the SQL semantics of $\text{COUNT}(\ast)$), the aggregation operator $\gamma_{G,f(A)}$ groups its input by G and computes an aggregate $f \in \{\text{COUNT}, \text{SUM}, \text{AVG}, \dots\}$ over A within each group, returning one tuple per distinct group value. Applied to the join result $J(D)$, the *join-aggregate* query result is

$$J_{\text{agg}}(D) := \gamma_{G,f(A)}(J(D)). \quad (1)$$

If $G = \emptyset$, $J_{\text{agg}}(D)$ collapses to a single scalar.

We use OUT to denote the size of the final query result returned to the verifier, and express all complexity bounds in this paper in IN and OUT . For a plain join, $OUT = |J(D)|$ is the number of join tuples. For a join-aggregate query, $OUT = |J_{\text{agg}}(D)|$ is the number of distinct groups, which can be orders of magnitude smaller than $|J(D)|$ since

¹ If the data size is itself sensitive, the prover can pad each relation with dummy tuples and publish only the padded size.

many join tuples may collapse into the same group. A central feature of our design is that the proof cost scales with the *final OUT* rather than the full join size $|J(D)|$, even when the two differ substantially.

3.2 Zero-Knowledge Proofs

Zero-knowledge (ZK) proofs [22] allow a prover to convince a verifier that a public statement is true without revealing any additional information beyond the validity of the statement itself. In this work, we use *zero-knowledge Succinct Non-interactive Arguments of Knowledge* (ZK-SNARKs) [19, 24], which provide three standard guarantees:

- *Completeness*. An honest prover can prove a true statement.
- *Soundness*. A dishonest prover cannot prove a false statement except with negligible probability.
- *Zero-knowledge*. The verifier learns nothing beyond the statement.

At a high level, a ZK-SNARK proves that there exists a private witness satisfying a public set of constraints for a given public statement. The witness consists of the hidden information needed to justify why the statement is true; depending on the application, it may include secret inputs, intermediate values, or an execution trace of the computation being certified. To enable verification, the target computation is encoded as an arithmetic circuit. The circuit is a public object that specifies how the witness and the public statement must relate. In practice, its constraints are represented as low-degree polynomial equations, each of which evaluates to zero if and only if the witness corresponds to a valid execution of the computation. The prover then generates a proof that there exists a witness satisfying all these constraints, without revealing the witness itself. This is just an abstract description of a general ZK proof implementation. In the next section, we describe in detail how to implement this process for our problem setting.

4 Problem Definition

In this work, we focus on queries Q consisting of multi-way joins, predicates, projection, and aggregation applied to the join result. Our target is to propose a ZK proof implementation that verifies multi-way join queries in a privacy-preserving manner.

4.1 Workflow

To solve the above problem, we handle each query through the following workflow. Here, we use the ZK-SNARK system, and our target is to prove that the claimed query result was computed correctly from a given input database.

Step 1 Data Commitment: Before answering any queries, the prover publishes a cryptographic commitment using the Inner Product Argument (IPA) protocol [10], $\text{Commit}(D)$, which binds all subsequent proofs to a single input database. The details are presented in Appendix A.

Step 2: Proof Generation: When query Q arrives, the prover publishes $Q(D)$. At the same time, the prover generates and publishes a proof showing that the claimed result was derived from the database committed to by $\text{Commit}(D)$. This verification is typically not a single-step process. Instead, it often depends on additional intermediate values that help link the committed database to the claimed final output. These intermediate values, together with the private input database, are collectively called the witness. Importantly, the witness must remain hidden from the verifier. To make verification possible, the intended computation is encoded as an arithmetic circuit

which specifies the relationships among the witness, the committed database, and the claimed result. The prover then generates a proof that there exists a private witness such that all circuit constraints are satisfied with respect to the committed database and the claimed output, without revealing the witness itself. In practice, these constraints are represented as low-degree polynomial equations, each of which evaluates to zero if and only if the witness corresponds to a correct execution of the computation that produces the claimed result. This can be done by running an existing ZK-SNARK system, specifically Halo2 [50], over the designed circuit and witness.

Step 3: Proof Verification: By checking that these equations are satisfied, the verifier can efficiently validate the correctness of the claimed result with logarithmic running time.

4.2 Threat Model and Security Guarantees

We consider a mutually distrustful setting between the prover and the verifier, which gives rise to three threats: (i) the prover may answer queries using a fabricated or modified database rather than the one it originally committed to; (ii) the prover uses the committed database but returns an incorrect query result; and (iii) a curious verifier may attempt to infer information about the private database beyond what is intentionally revealed by the query result.

Threat (i) is addressed by the database commitment: every subsequent proof is generated with respect to $\text{Commit}(D)$, preventing the prover from silently switching to a different database without causing verification to fail. Threat (ii) is addressed by the standard security properties of ZK-SNARKs, which are inherited by our Halo2 [50] instantiation [11, 51]. To defend against Threat (iii), since the circuit is publicly visible, we require it to be oblivious, meaning that its size and structure should not leak any information about the database. Under this condition, once the proof is generated by ZK-SNARKs (Halo2 in our implementation), the zero-knowledge property of ZK-SNARKs [22] guarantees that the proof will not leak any information besides the query results.

4.3 Problem Reduction

We now analyze the running time of the above workflow. The prover can compute the data commitment (**Step 1**) in linear time [19], and the verifier can verify the proof (**Step 3**) in logarithmic time [12]; see Appendix A for the details. As a result, the main bottleneck lies in proof generation (**Step 2**). In this step, when the prover uses Halo2 [50] to generate a proof over the circuit and witness, the running time is roughly linear in the circuit size. In this paper, we treat the underlying cryptographic procedures of generating proofs from circuits as black boxes and defer the details to Appendix A. As a result, the remaining task is to design a circuit that is (1) oblivious and (2) efficiently encodes the relationship between the input and the claimed output.

Designing such a circuit for multi-way joins, however, is non-trivial. Existing ZK database systems decompose a multi-way join into a sequence of binary joins and include *every* intermediate join result in the witness. Because each witness value must occupy circuit cells and be enforced by constraints, witness size and circuit size are tightly coupled. Therefore, reducing proving time requires reducing the witness size as well, which turns the problem into designing an oblivious circuit that avoids materializing large intermediate join results.

5 ZK Proof Primitives

With the problem setting and privacy model established, we now describe the ZK proof primitives that serve as building blocks for our join-verification circuits. Our cryptographic backend is Halo2, whose setup and proof-complexity model are detailed in Appendix A.

5.1 PLONKish Arithmetization

We briefly review PLONKish arithmetization; see [26] for details and Appendix B for an example. A PLONKish circuit is a rectangular matrix with three column types: *fixed columns*, which store circuit-defined constants such as selectors; *advice columns*, which hold private witness values assigned by the prover; and *instance columns*, which contain public inputs and outputs. Because our join-verification logic mainly uses advice columns, we write **columns** to mean advice columns unless stated otherwise.

Computation is enforced by two kinds of constraints. *Gate constraints* are low-degree polynomials over the columns that must vanish on each row; selector columns activate different gate types on different rows, allowing heterogeneous operations in one circuit. *Copy constraints* enforce equality across cells via a permutation argument [19], linking outputs of one gate to inputs of another. The prover assigns the advice columns to form an *execution trace*, and the proof certifies that all constraints hold without revealing the trace.

We use *gate* in a slightly broader sense than a single constrained row: a gate is a reusable *sub-circuit* implementing one logical operation, such as a range check, lookup, or semijoin check. Larger circuits are built by composing such gates, and total circuit size is the sum of the costs of all instantiated gates.

REMARK 1. A ZK-SNARK circuit does not do computation. The prover computes it off-circuit and supplies it as private witness, and the circuit only verifies that the input, the witness, and the claimed output are consistent. Every gate in this paper is stated in that form, so its cost is the cost of checking a claim rather than of producing it.

5.2 Lookup Arguments and Derived Gates

Lookup arguments [18, 27] allow proving that every element of a private witness column P belongs to a public table Q . In this paper, we use the Plookup protocol [18], whose core is a *permutation argument*: a proof that one column of n rows is a reordering of another. Three steps make this argument certifiable in $O(n)$ constraints without revealing the reordering.

Supplying the reordering. By Remark 1, the prover computes the reordered column P' off-circuit and supplies it as a witness column. Representing the permutation σ itself would take $\Theta(n \log n)$ information, and *generating* P' in-circuit is what would cost that much, whereas merely *linking* P and P' does not.

Linking P and P' by a grand product. A permutation maps row i to a *different* row $j = \sigma(i)$, so no row-by-row identity can link the two columns. A grand product removes the row indices from the picture: two size- n columns are reorderings of each other exactly when $\prod_i (X + P_i) = \prod_i (X + P'_i)$ as polynomials in X , since the product ranges over *all* rows and is invariant to the order of its factors. Checking this identity at a random challenge X costs two length- n grand products, i.e., $O(n)$ constraints, and by the Schwartz–Zippel lemma [40] it fails with probability at most $n/|\mathbb{F}|$, where $\mathbb{F}$ is the 255-bit scalar field our circuits are defined over. This is the standard

shuffle argument [8, 35]: the same identity underlies the permutation and lookup techniques of PLONK [19], Plookup [18], Halo2 [50], and the prior ZK database systems vSQL [53] and ZKSQL [32].

From multiset equality to subset. The subset predicate that lookups need, $P \subseteq Q$, adds one constraint per row. In addition to P' the prover supplies Q' , a reordering of the table Q , again off-circuit; both columns occupy the same circuit height n , the lookup column and the table each padded with a reserved sentinel, the form Halo2's lookup argument uses [50]. The grand product above extends to both pairs at once,

$$\prod_i (P_i + \alpha)(Q_i + \beta) = \prod_i (P'_i + \alpha)(Q'_i + \beta), \quad (2)$$

where i ranges over all row indices and α, β are random challenges, which certifies that P' and Q' are reorderings of P and Q . What remains is the lookup relationship itself. The prover arranges P' with equal values adjacent and aligns Q' to it, so that the first occurrence of each value in P' faces a matching table entry, and the circuit imposes $P'_0 = Q'_0$ and $(P'_i - Q'_i)(P'_i - P'_{i-1}) = 0$ for every i . At each row the second constraint forces either $P'_i = P'_{i-1}$, the value repeats the preceding one, or $P'_i = Q'_i$, a newly occurring value matches an entry of Q' ; starting from $P'_0 = Q'_0$, every distinct value of P' , and hence every value of P , therefore appears in Q , at $O(1)$ per row.

For large domains (e.g., 64-bit integers), a *decomposed* strategy splits each value into $t = \lceil B/b \rceil$ chunks of b bits and checks each chunk against a small table of size 2^b , making the number of constraints independent of the domain size 2^B . Building on lookup arguments, we employ two derived gates used throughout this work:

- **Range Check** [25, 26]. Verifies that a value in a column lies within $[0, 2^B)$ using the decomposed lookup strategy described above. Each value is split into $t = \lceil B/b \rceil$ chunks of b bits, and each chunk is checked against a lookup table of size 2^b .
- **Sort** [26]. Verifies that the values in a column are in non-decreasing order by computing consecutive differences $d_i = x_{i+1} - x_i$ and applying a range check to enforce $d_i \geq 0$.

These primitives serve as the foundation for the Oblivious Join Gate presented in Section 6.

6 OBJ: Efficient Verification for Acyclic Joins

The core challenge for join verification is to certify the query result without materializing any intermediate join inside the circuit, since each materialized intermediate must be padded to its worst-case bound to achieve obliviousness, leading to prohibitive circuit sizes of $O(IN^k)$ for a k -way join. Our insight is that an acyclic multi-way join can be decomposed via semijoin reductions along a join tree, and each reduction can be verified independently with a linear-cost circuit check. Rather than re-executing the join, the prover splits each input table into clean tuples (those participating in the join result) and residual tuples (those that do not), and the circuit verifies this split through lightweight structural checks without materializing intermediate join results.

In this section, we propose Oblivious Join Gate (OBJ), which achieves efficient verification for acyclic joins, and build it up from the local building block to the full query. We begin in Section 6.1 with a *semijoin gate*: for a single edge (R_i, R_j) of the join tree, it certifies that the prover has correctly separated R_i into clean and residual tuples with respect to R_j . This gate has linear cost $O(IN)$ and serves as the basic module for the rest of the section. Building on it, Section 6.2 states

what an OBJ must certify and gives a direct *Two-Pass OBJ* that is similar to the classical bottom-up/top-down semijoin reduction [9, 48]. We then presents a more efficient *One-Pass OBJ* (Section 6.3), in which the prover directly supplies the final clean/residual split so the circuit only has to certify it, on two building blocks introduced there, indexed relations and multiplicity propagation. Section 6.4 computes aggregates over the clean relations by reusing the multiplicity propagation, and Section 6.5 covers the final join enumeration for plain joins, which reuses both building blocks.

6.1 Semijoin Gate

We start with the fundamental building block of OBJ: a gate that certifies, for a single join-tree edge, that the prover's claimed partition of a relation into clean and residual tuples is correct. Once this linear-cost gate is available, the full OBJ is obtained by composing one instance per tree edge.

Consider one edge (R_i, R_j) of a join tree with shared join attributes $K_{ij} = \text{attr}(R_i) \cap \text{attr}(R_j)$. The prover claims that two witness tables R'_i and R''_i realize the exact semijoin/antijoin split of R_i with respect to R_j :

$$R'_i = R_i \ltimes R_j, \quad R''_i = R_i \triangleright R_j. \quad (3)$$

The gate should certify this claim without recomputing the semijoin inside the circuit. We specify each gate in this paper by three factors: the *private witness* (what secret data the prover supplies), the *output* (what the gate certifies for downstream use within the circuit), and the *verification target* (what property the circuit constraints enforce).

DEFINITION 1 (SEMIJOIN GATE). Fix one directed edge (R_i, R_j) of a join tree T and its shared join attributes K_{ij} . We define the semijoin gate $\text{SJ}_{i \rightarrow j}$ through the following three factors:

- **Private witness.** The tuples of R_i and R_j , together with the prover-supplied partition (R'_i, R''_i) .
- **Output.** A certified partition (R'_i, R''_i) , where R'_i is the kept part and R''_i is the residual part. This output is usually consumed by later gates rather than exposed directly to the verifier.
- **Verification target.** The gate certifies (3), i.e., that (R'_i, R''_i) is exactly the semijoin/antijoin split of R_i with respect to R_j . Equivalently, the witness must satisfy three semantic conditions: (i) conservation of R_i across (R'_i, R''_i) , (ii) membership of R'_i 's keys in R_j , and (iii) non-membership of R''_i 's keys in R_j , formalized respectively as:

$$R_i \equiv R'_i \cup R''_i, \quad (4)$$

$$\pi_{K_{ij}}(R'_i) \subseteq \pi_{K_{ij}}(R_j), \quad (5)$$

$$\pi_{K_{ij}}(R''_i) \cap \pi_{K_{ij}}(R_j) = \emptyset. \quad (6)$$

Design idea. Our target is to certify $R'_i = R_i \ltimes R_j$ and $R''_i = R_i \triangleright R_j$ without ever computing the semijoin inside the circuit. Our insight is that these two equalities jointly decompose into three weaker structural conditions that together imply them: conservation of R_i (no tuples lost or fabricated), membership of R'_i 's keys in R_j , and non-membership of R''_i 's keys in R_j . Each of these is linear-cost to check, and together they leave no cheating strategy: a joinable tuple hidden in R''_i would violate non-membership, a fabricated tuple in R'_i would violate conservation, and a non-joinable tuple in R'_i would violate membership. The gate therefore reduces two quadratic-looking equality checks to three linear-cost structural checks.

Realization. The prover locally computes the partition (R'_i, R''_i) and supplies it directly as witness columns. Since the partition is exhaustive, the combined size $|R'_i| + |R''_i| = |R_i|$ is determined solely by the public input size. The circuit then enforces the three verification conditions (4), (5), and (6) from the Verification target of Definition 1 using three checks.

- **Conservation Check** (permutation argument) realizes (4). By treating the concatenation $R'_i \cup R''_i$ as a single virtual column and checking via the permutation argument ((2)) that it is a permutation of R_i , the circuit enforces multiset equality, guaranteeing that no tuples are fabricated or dropped. This check contributes $O(IN)$ constraints.
- **Membership Check** (lookup argument) realizes (5). Treating the key column of R'_i as the witness and the key column of R_j as the reference table, the lookup argument enforces $\pi_{K_{ij}}(R'_i) \subseteq \pi_{K_{ij}}(R_j)$, proving that every kept tuple has a matching key in R_j . This check contributes $O(IN)$ constraints.
- **Non-Membership Check** (merge-and-sort) realizes (6). A lookup argument cannot directly prove that a value is *not* in a table, so we use a merge-and-sort strategy: (1) the prover merges both key columns into one and provides a sorted permutation of the merged sequence as witness; (2) a permutation argument verifies that the sorted sequence is a rearrangement of the merged keys; (3) a sort gate verifies non-decreasing order; and (4) a local gate constraint checks that no two adjacent elements from different sources are equal. If the sorted merged sequence has no equal adjacent pair from the two sources, the key sets must be disjoint. This check contributes $O(IN)$ constraints.

Since lookups and permutation arguments operate on single-column values, multi-attribute tuples are first compressed to one field element per tuple. Putting the pieces together, the semijoin gate has linear cost $O(IN)$: the witness layout allocates $|R_i|$ rows at no constraint cost, and each of (4), (5), (6) contributes $O(IN)$ constraints. The detailed constant-factor breakdown appears in Appendix G.2.

6.2 From Semijoin Verification to Two-Pass OBJ

With the semijoin gate in hand, we can now move from local verification to whole-query verification. The goal is no longer just to certify one local split $R_i = R'_i \cup R''_i$ on a single edge, but to certify that the collection of all reduced (“clean”) relations across the whole join tree forms the fully reduced instance of the acyclic query. OBJ accomplishes this by arranging semijoin checks along a join tree and treating the clean relations as a compact witness for the final result.

DEFINITION 2 (OBLIVIOUS JOIN GATE (OBJ)). Fix an acyclic join query together with a join tree T . We define the Oblivious Join Gate through the following three factors:

- **Private witness.** The input relations $\mathcal{R} = \{R_1, \dots, R_k\}$, together with prover-supplied auxiliary data. We will present two realizations of the OBJ, each with its own witness layout, so the detailed witness is left abstract here and specified per realization.
- **Output.** The certified clean instance $\mathcal{R}^c = \{R_1^c, \dots, R_k^c\}$, where each $R_i^c \subseteq R_i$ contains exactly the tuples of R_i that participate in the global join; by complement, the residual relations are $\mathcal{R}^r = \{R_1^r, \dots, R_k^r\}$ with $R_i^r = R_i \setminus R_i^c$. As with the semijoin gate, $\mathcal{R}^c$ is consumed within the circuit: the final join result or aggregate is computed from $\mathcal{R}^c$ after the gate has certified it.

- **Verification target.** The gate certifies that $\mathcal{R}^c$ is the fully reduced instance of the acyclic query: it removes every dangling tuple and keeps exactly those tuples that can be extended to at least one global join result. Since we will present two realizations of the OBJ, each with its own set of concrete verification conditions, we specify these conditions per realization rather than here.

The definition fixes only the input/output contract: given $\mathcal{R}$, the gate must output a clean instance $\mathcal{R}^c$ that equals the fully reduced instance of the query. It does not prescribe how $\mathcal{R}^c$ is produced or how many semijoin gates are invoked internally; any circuit that certifies this contract qualifies as an OBJ realization. Semijoin gates supply the local edge-level checks, and the realization chooses how to compose them along T . This subsection and the next present two such realizations: a direct Two-Pass OBJ that closely follows the classical bottom-up/top-down semijoin reduction along the join tree T , and a more efficient One-Pass OBJ that takes advantage of prover-supplied final clean/residual partitions.

6.2.1 Two-Pass OBJ. We start from the classical semijoin reduction [48] for acyclic queries. Given a join tree T with an arbitrary root, the algorithm performs two in-place semijoin passes: a *bottom-up* pass that, for each edge (P, C) processed from leaves to root, replaces the parent by $P \leftarrow P \bowtie C$, so that after the pass the root retains only tuples with a matching combination in every descendant subtree; and a *top-down* pass that, for each edge (P, C) processed from root to leaves, replaces the child by $C \leftarrow C \bowtie P$ on the already-reduced parent, so that every relation retains only tuples extendable to at least one global join result. We adapt this algorithm to the ZK setting by replacing each in-place semijoin with one invocation of the semijoin gate of Section 6.1. Concretely: (i) for every tree edge we invoke a semijoin gate in the bottom-up pass and another in the top-down pass, yielding $2(k-1)$ gate invocations that jointly certify the clean relations $\{R_1^c, \dots, R_k^c\}$; and (ii) a final *pairwise consistency* check across all edges certifies that the clean relations agree on their shared keys, ruling out the corner case where two edges are each individually consistent but jointly inconsistent. The circuit never recomputes a semijoin; it only verifies prover-supplied partitions edge by edge.

Realization. We compose semijoin gates along all edges of the join tree T to verify the full acyclic join $R_1 \bowtie \dots \bowtie R_k$ in two passes.

In the *bottom-up pass*, we process edges from leaves to root, applying a semijoin gate for each edge (P, C) to reduce the parent: $P \leftarrow P \bowtie C$. After processing all children of a node, that node retains only tuples that have a match in every descendant subtree. For instance, in a tree rooted at R_2 with children R_1 and R_3 (as in Example 6.1), the bottom-up pass reduces R_2 by R_1 and then by R_3 , yielding the reduced root R_2^c .

In the *top-down pass*, starting from the fully reduced root, we propagate the reduction outward. For each edge (P, C) processed from root to leaves, we apply a semijoin gate to reduce the child: $C \leftarrow C \bowtie P$, where P is the already-reduced parent. This filters each relation by its reduced parent, ensuring global consistency across all tree edges. Across the two passes, every input relation R_i is split into the final clean part R_i^c and the global residual $R_i^r = R_i \setminus R_i^c$; R_i^r aggregates all tuples of R_i discarded by the semijoin gates on edges incident to R_i .

After both passes produce the fully reduced relations $\{R_1^c, R_2^c, \dots, R_k^c\}$, a pairwise consistency check verifies that adjacent clean relations agree on their shared join keys. For every edge (R_i, R_j) with shared

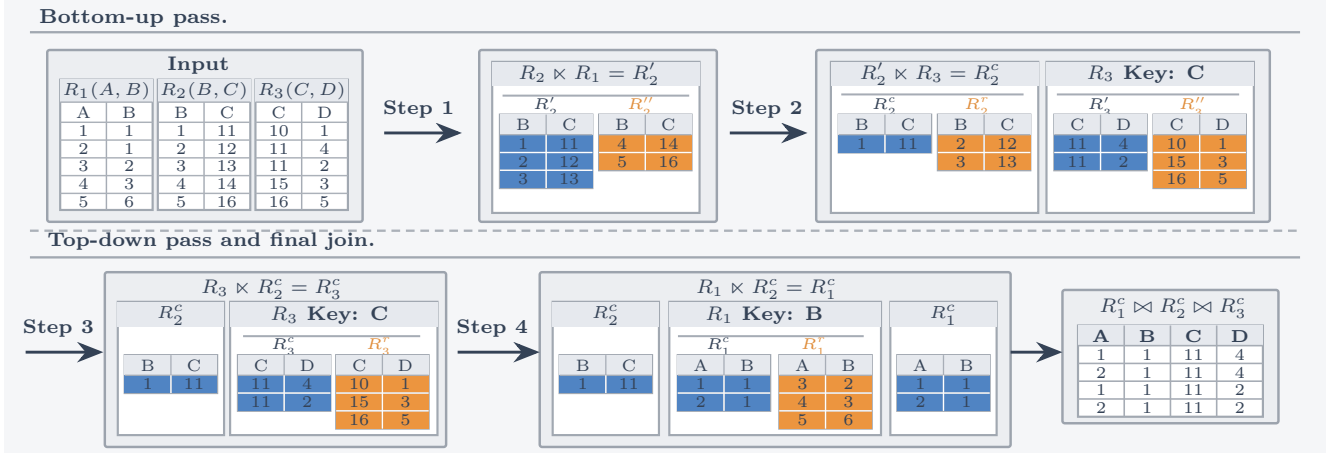


Figure 1: Illustration of the Two-Pass OBJ on the SQL query $\text{SELECT } * \text{ FROM } R_1, R_2, R_3 \text{ WHERE } R_1.B=R_2.B \text{ AND } R_2.C=R_3.C$.

attributes K_{ij} , the circuit checks $\pi_{K_{ij}}(R_i^c) = \pi_{K_{ij}}(R_j^c)$ via two Membership Checks ($R_i^c \rightarrow R_i^c$ and $R_j^c \rightarrow R_i^c$). This certifies that every clean tuple has a matching join partner in every adjacent clean relation, ensuring the reduced instance admits no dangling tuples.

Finally, the output is computed by joining the verified clean relations along the tree. Verifying that the claimed output matches this join is deferred to Section 6.5. We defer the detailed correctness and obliviousness analyses of both OBJ variants to Appendix F.

Example 6.1 (Two-Pass OBJ on a 3-way path join). Consider the acyclic join $Q = R_1(A, B) \bowtie R_2(B, C) \bowtie R_3(C, D)$, with join predicates $R_1.B = R_2.B$ and $R_2.C = R_3.C$ (as visualized in Figure 1). Let the input relations be R_1, R_2 and R_3 .

With the join tree rooted at R_2 (so R_1 and R_3 are its children), we begin with the bottom-up pass to reduce each parent relation against its children along tree edges, keeping only tuples whose join keys have a matching combination in every descendant subtree. For each edge, the prover supplies a kept witness $R_i \bowtie R_j$ and a residual witness $R_i \triangleright R_j$, and the circuit enforces conservation via a permutation argument, membership for kept keys via lookup, and non-membership for residual keys. Specifically, the following two steps are performed.

Step 1: Process edge $(P, C) = (R_2, R_1)$, reducing parent R_2 against child R_1 on key B. Since $\pi_B(R_1) = \{1, 2, 3, 6\}$, the tuples $(4, 14)$ and $(5, 16) \in R_2$ (with $B=4, 5$) cannot participate in the join with R_1 and are discarded. The semijoin gate partitions $R_2 = R'_2 \cup R''_2$:

$$R'_2 = R_2 \bowtie R_1 = \{(1, 11), (2, 12), (3, 13)\}, \quad R''_2 = R_2 \triangleright R_1 = \{(4, 14), (5, 16)\}.$$

The circuit checks: (1) $R_2 \equiv R'_2 \cup R''_2$ (conservation via permutation), (2) $\pi_B(R'_2) \subseteq \pi_B(R_1)$ (membership via lookup), (3) $\pi_B(R''_2) \cap \pi_B(R_1) = \emptyset$ (non-membership). These three checks are performed similarly at the end of the next three steps and are omitted there for brevity.

Step 2: Process edge $(P, C) = (R_2, R_3)$, reducing parent R'_2 against child R_3 on key C. Now $\pi_C(R'_2) = \{11, 12, 13\}$ and $\pi_C(R_3) = \{10, 11, 15, 16\}$ intersect only on $C=11$, so R'_2 is split into $R^c_2 = R'_2 \bowtie R_3 = \{(1, 11)\}$ and a local discard $R'_2 \triangleright R_3 = \{(2, 12), (3, 13)\}$. Aggregating with Step 1 gives the global residual for R_2 :

$$R^c_2 = R_2 \setminus R'_2 = R''_2 \cup (R'_2 \triangleright R_3) = \{(2, 12), (3, 13), (4, 14), (5, 16)\}.$$

After this step the root R^c_2 is fully reduced, while the children R_1 and R_3 remain untouched; the top-down pass will reduce them against R^c_2 .

With R^c_2 fully reduced, we begin the top-down pass to propagate its keys outward and reduce each child against the already-reduced parent, so that every remaining child tuple can be extended to at least one global join result. This includes the following Steps 3 and 4.

Steps 3 and 4: reduce each child against the reduced parent R^c_2 . Since $\pi_C(R^c_2) = \{11\}$ and $\pi_B(R^c_2) = \{1\}$, only the tuples of R_3 with $C=11$ and of R_1 with $B=1$ survive:

$$R^c_3 = \{(11, 4), (11, 2)\}, \quad R^c_1 = \{(10, 1), (15, 3), (16, 5)\}, \\ R^c_1 = \{(1, 1), (2, 1)\}, \quad R^c_1 = \{(3, 2), (4, 3), (5, 6)\}.$$

After the two passes, the fully reduced (clean) instance is:

$$R^c_1 = \{(1, 1), (2, 1)\}, \quad R^c_2 = \{(1, 11)\}, \quad R^c_3 = \{(11, 4), (11, 2)\}.$$

As the final step of the Two-Pass OBJ, the circuit verifies edge consistency via mutual lookups: $\pi_B(R^c_1) = \{1\} = \pi_B(R^c_2)$ and $\pi_C(R^c_2) = \{11\} = \pi_C(R^c_3)$. At this point $\{R^c_1, R^c_2, R^c_3\}$ is certified as the fully reduced instance of Q , and the final join is evaluated from it: $Q(D) = R^c_1 \bowtie R^c_2 \bowtie R^c_3$, yielding $|Q(D)| = 2 \cdot 1 \cdot 2 = 4$ tuples, matching Figure 1. ■

6.3 Efficient One-Pass OBJ

The Two-Pass OBJ reconstructs the reduced instance inside the circuit by invoking $2(k-1)$ semijoin gates, traversing every tree edge once bottom-up and once top-down. Our insight is that the prover already knows the final answer before the circuit runs, so the reconstruction can be done *outside* the circuit: the prover computes the clean/residual split (R^c, R') off-circuit using any efficient method (for instance the classical two-pass semijoin reduction on the database side) and supplies it directly as a witness. The circuit's task therefore shrinks from *producing* the reduction to *certifying* a prover-supplied one.

Certifying a prover-supplied partition reduces to three intuitive questions: (i) Does (R^c, R') actually partition R_i , with no tuple fabricated, lost, or double-counted? (ii) Is the clean side a coherent join skeleton, i.e., does every clean tuple have matching join partners in every adjacent clean relation? (iii) Was any joinable tuple hidden in the residual side? Questions (i) and (ii) decompose into local per-relation and per-edge checks. Question (iii) looks harder because a hidden joinable tuple is a global witness: it sits in R^c , but its join partners stay

clean, so no check confined to $\mathcal{R}^c$ can expose it. Counting exposes it instead: the join of the clean parts is contained in the join of the input relations, so equal row counts force the two to be equal as multisets, and a tuple hidden in a residual part would leave the clean join strictly smaller. Both counts come from one bottom-up pass along the join tree, one channel each; the input-side count serves only this comparison, and everything the query returns is read from the clean relations.

Two building blocks answer (i) and (iii), and the operators that consume $\mathcal{R}^c$ reuse both: *indexed relations*, over which the partition is claimed per occurrence so that one Conservation Check certifies it, and the *multiplicity propagation*, which annotates every tuple with the number of join rows it takes part in, computed bottom-up along the join tree without enumerating the join, so that one traversal yields the two counts that (iii) compares. We introduce them below.

Indexed Relations. To name occurrences rather than tuple values, we index the rows of each relation by their committed position,

$$\widehat{R}_i = \{(\ell, t_\ell, c(t_\ell)) : \ell \in [R_i]\},$$

where t_ℓ is the tuple stored in row ℓ , the index ℓ itself is carried by a fixed column, and $c(t) \in \{0,1\}$ is the *clean indicator*, marking the row holding t as clean, one indicator per row; the indices are distinct.

Multiplicity Propagation. The key technical idea is to propagate an auxiliary per-tuple count along the join tree, following the annotated-relations paradigm [4]; we call this mechanism the *multiplicity propagation*. The count wanted at a root tuple is global, since it depends on how many tuples in every descendant subtree match it on shared keys, so propagation assembles it bottom-up as a product-of-sums over the children's already-established counts, without enumerating the join.

We call this per-tuple count the *multiplicity* of a tuple t and denote it $\mu(t)$, so $\mu(t)$ is the number of join rows that extend t through the subtree below its relation, at the root the number of rows of $R_1^c \bowtie \dots \bowtie R_k^c$ to which t contributes. The prover supplies $\mu(t)$ for every row of every relation as part of the auxiliary witness. Leaf-relation multiplicities are trivially 1, and internal-node multiplicities encode the number of join extensions into the subtrees below. Concretely, for each tree edge (R_i, R_j) where R_i is the parent and R_j is the child, after all children of R_i have been processed, each tuple $t_i \in R_i^c$ satisfies

$$\mu(t_i) = \prod_{R_j \text{ child of } R_i} \left(\sum_{\substack{t_j \in R_j^c \\ \pi_{K_{ij}}(t_j) = \pi_{K_{ij}}(t_i)}} \mu(t_j) \right),$$

where K_{ij} denotes the shared join attributes on edge (R_i, R_j) . This equation is a *recurrence* that correct multiplicities must satisfy, not a fresh definition: the semantic meaning of μ was already fixed. The recurrence follows from two observations: (1) the extensions of t_i into different child subtrees combine independently, hence the product over children; and (2) within a single child subtree rooted at R_j , the extensions of t_i are exactly those child tuples $t_j \in R_j^c$ that agree with t_i on the shared key K_{ij} , each already carrying its own count of downstream extensions via $\mu(t_j)$, hence the inner sum. Combined with the leaf base case $\mu(t) = 1$, the recurrence uniquely determines the correct μ at every node, and the root sum is the size of the join it runs over.

The circuit verification of multiplicity propagation establishes the recurrence edge by edge, traversing the join tree bottom-up from the leaves so that each parent's equality check is made only after all

its children's edges have been processed. For each edge (R_i, R_j) it performs three checks, each enforcing a specific property needed to pin down the recurrence's inner sum $\sigma_j(t_i)$:

- (1) *Contiguity of matching children.* Sort R_j^c by K_{ij} using the sort gate (Section 5.2), so child rows sharing a K_{ij} value become adjacent in the sorted column. This step turns the set-of-matches condition $\pi_{K_{ij}}(t_j) = \pi_{K_{ij}}(t_i)$ into a contiguous-range condition that downstream gates can scan in a single pass.
- (2) *Correct per-key aggregation.* On the sorted R_j^c , a group-by followed by an aggregation gate (reused from PoneglyphDB [26]) produces, for every distinct value v of K_{ij} , the sum $\sigma_j(v) = \sum_{t_j: \pi_{K_{ij}}(t_j)=v} \mu(t_j)$. The output is a key-indexed table $(v, \sigma_j(v))$.
- (3) *Correct parent-side combination.* For each $t_i \in R_i^c$, a lookup argument on K_{ij} retrieves $\sigma_j(t_i[K_{ij}])$ from the table above, and a multiplication gate folds it into a running product maintained per t_i . Once all children of R_i have been processed, a single equality constraint compares this accumulated product against the prover-supplied $\mu(t_i)$. This lookup is the membership test a semijoin gate performs on the same edge, returning the number of matching child tuples rather than a bit, which is why the One-Pass OBJ below needs no semijoin gate of its own.

Correctness follows: if all three checks pass on every edge, then every $\mu(t_i)$ equals the product-of-sums value mandated by the recurrence; combined with the leaf base case $\mu(t) = 1$, this forces the prover's annotations to coincide with the unique correct multiplicities.

Realization. The partition is claimed at the level of occurrences: the prover sets the clean indicator c on every row of the indexed relations $\widehat{R}_i$. The circuit then certifies $\mathcal{R}^c$ as the fully reduced instance by enforcing three conditions:

$$\widehat{R}_i \equiv \widehat{R}_i^c \uplus \widehat{R}_i^r, \quad \forall i \in [k], \quad (7)$$

$$\pi_{K_{ij}}(R_i^c) = \pi_{K_{ij}}(R_j^c), \quad \forall (R_i, R_j) \in T, \quad (8)$$

$$\sum_{t \in R_{\text{root}}} \mu^c(t) = \sum_{t \in R_{\text{root}}} \mu^{\text{all}}(t). \quad (9)$$

Because every index occurs once in $\widehat{R}_i$, (7) says in a single statement that no occurrence is fabricated, lost, duplicated, or placed on both sides (the clean side satisfies the query's in-relation predicates, as explained in Appendix E). In (9), t ranges over every row of R_{root} , and μ^c and μ^{all} are the multiplicities of the propagation above, run over the clean parts and over the predicate-filtered input and selected on the committed rows by their *anchors*, the clean indicator c and the certified predicate bit b , the constant 1 on a relation carrying no predicate; a residual row thus contributes 0 on the clean side, and a predicate-failing row contributes 0 to both. We describe how to realize the circuit for these three conditions.

- **Conservation Check.** (7) is realized by a single Conservation Check (a permutation argument) per relation, between $\widehat{R}_i$ and the two parts laid end to end in one virtual column, which the permutation reads as the multiset $\widehat{R}_i^c \uplus \widehat{R}_i^r$.
- **Pairwise Consistency Check.** (8) requires the clean sides on every tree edge to project to the same key set, so every clean tuple is extendable along each adjacent edge. Each edge's set equality is realized by two mutual Membership Checks run directly between the key columns, gated by the clean indicator on both sides: each row contributes $c(t)(t[K_{ij}] + 1)$, so a residual row reads 0.

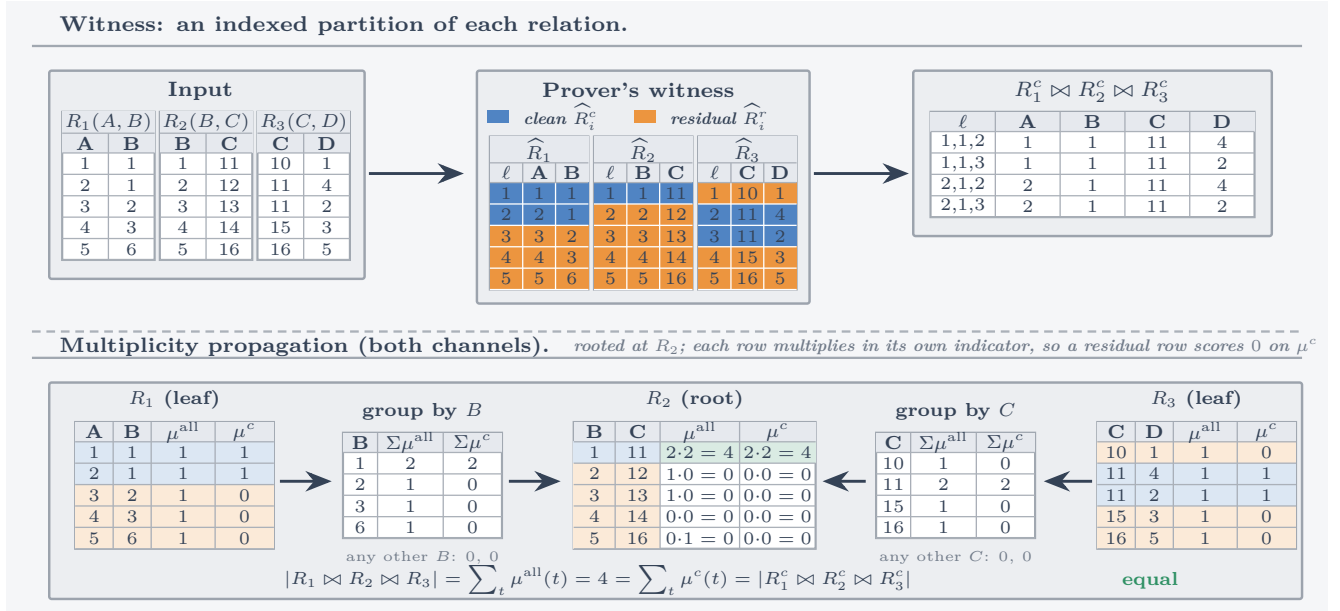


Figure 2: Illustration of the One-Pass OBJ on the SQL query $\text{SELECT } * \text{ FROM } R_1, R_2, R_3 \text{ WHERE } R_1.B = R_2.B \text{ AND } R_2.C = R_3.C$.

- **Cardinality Preservation Check.** The two channels of (9) are two instances of the multiplicity propagation, anchored at c and at b , and share one traversal of the join tree. Per edge, the merged key columns of R_i and R_j are sorted, one scan emits the per-key pair $(\sigma^{\text{all}}, \sigma^c)$, and each parent row looks the pair up and folds it into its two running products; residual and predicate-failing rows need no special handling, since their anchors, multiplied in at every node, zero their contributions. Neither join is materialized and neither sum is exposed.

Together, (7)–(9) certify $\{R_1^c, \dots, R_k^c\}$ as the fully reduced instance. Soundness rests on two facts: (8) gives every clean tuple partners on every edge, so no clean tuple is spurious, and (9) forces the clean join to be the whole input join, so no participating occurrence is missing; (7) is what lets both be read off one partition. A detailed correctness proof is given in Appendix F.2 alongside the obliviousness analysis.

Example 6.2 (One-Pass OBJ on a 3-way path join). Consider the same query and data as in Example 6.1 (Figure 2). With the join tree rooted at R_2 , the prover supplies the indexed full clean/residual split directly as witness. The circuit then certifies this witness by enforcing the three conditions (7)–(9) in a single pass. Specifically, the following three steps are performed.

Step 1. For each relation, the circuit checks (7) via a permutation argument over the indexed occurrences, for instance

$$\hat{R}_2 \equiv \{(1, (1, 11))\} \cup \{(2, (2, 12)), (3, (3, 13)), (4, (4, 14)), (5, (5, 16))\}$$

so no tuple of $\hat{R}_2$ is fabricated, dropped, or duplicated.

Step 2. (8) demands that on every tree edge the two clean sides agree on the shared join key,

$$\pi_B(R_1^c) = \{1\} = \pi_B(R_2^c), \quad \pi_C(R_2^c) = \{11\} = \pi_C(R_3^c),$$

so every clean tuple has a matching partner in every adjacent clean relation; residual rows contribute 0 on both sides.

Step 3. Finally, the Cardinality Preservation Check compares the two root sums, both produced by the single traversal drawn in Figure 2. Each leaf row starts at $\mu^{\text{all}} = 1$, and at $\mu^c = 1$ only when clean. Grouping R_1 by B gives $(\Sigma \mu^{\text{all}}, \Sigma \mu^c) = (2, 2)$ at $B = 1$ and $(1, 0)$ elsewhere, and grouping R_3 by C gives $(2, 2)$ at $C = 11$ and $(1, 0)$ elsewhere. At the root, $(1, 11)$ receives $2 \cdot 2 = 4$ on both channels while every other row of R_2 loses a factor, or its own indicator, and receives 0, so both root sums equal 4. Had the prover moved $(1, 1)$ to the residual part, the clean sum would have dropped to 2 while the input sum stayed at 4, and the equality would have failed.

Since all three conditions hold, $\{R_1^c, R_2^c, R_3^c\}$ is certified as the fully reduced instance of Q , and the final join is evaluated from it: $Q(D) = R_1^c \bowtie R_2^c \bowtie R_3^c$, yielding $|Q(D)| = 2 \cdot 1 \cdot 2 = 4$ tuples, matching Figure 2. ■

Complexity Analysis. Both OBJ variants achieve $O(IN)$ circuit complexity and differ only in their constant factors; the *OUT* term of the overall bound is contributed by whichever operator consumes $\mathcal{R}^c$. The Two-Pass OBJ instantiates $2(k-1)$ semijoin gates, one per tree edge in each pass, plus $2(k-1)$ Membership Checks for pairwise consistency. The One-Pass OBJ instantiates none: the top-down pass becomes k Conservation Checks over indexed occurrences, and the bottom-up pass becomes a single counting pass, the multiplicity propagation. Detailed per-gate cost derivations, total constant-factor comparisons, and the savings under uniform sizes appear in Appendix G.

6.4 Aggregation with One-Pass OBJ

The clean relations are an efficient input for the operators above them, allowing each to work on $\mathcal{R}^c$ with cost proportional to IN rather than to the full join size $|J(D)|$. Rather than materializing the full join result to compute aggregates, the prover attaches to each clean tuple an auxiliary count recording how many join rows the tuple participates in, and the circuit verifies these counts bottom-up along the join tree. The final aggregate is then computed as a single pass over the root relation's annotated tuples. For example, $\text{COUNT}(*)$ reduces

to summing these counts across the clean tuples of the root relation, which equals the total join size without ever enumerating the join. The same idea also yields SUM and AVG.

Realization. Now that the OBJ certifies the set of clean relations $\mathcal{R}^c = \{R_1^c, \dots, R_k^c\}$, the task is to compute the join-aggregate $J_{\text{agg}}(D) = \gamma_{G;f(A)}(J(D))$ (Equation (1)) without materializing $J(D)$. We focus on the common case of free-connex acyclic queries [6], i.e., those whose output attributes form a connected subset within the join tree; non-free-connex queries can be reduced to this case with an additional projection gate on the output attributes before aggregation.

The key technical idea is the multiplicity propagation of Section 6.3 run on the clean side, which gives every clean tuple its multiplicity $\mu(t)$ and 0 on every residual row; with the One-Pass OBJ these are the μ^c of (9). The aggregate then reduces to a read-off at the root.

Final Aggregation. After multiplicities have been propagated to the root relation R_{root}^c , the final aggregate reduces to a single weighted summation over R_{root}^c , but the exact form depends on where the aggregate attribute lives, so we split into two cases.

Case 1: the aggregate attribute is on the root. When the aggregated attribute A (or the wildcard $*$) is an attribute of the root relation, the aggregate is read off from R_{root}^c and its multiplicities via:

- **COUNT($*$):** $\sum_{t \in R_{\text{root}}^c} \mu(t)$
- **SUM(A):** $\sum_{t \in R_{\text{root}}^c} \mu(t) \cdot t[A]$
- **AVG(A):** $\text{SUM}(A) / \text{COUNT}(*)$

Case 2: the aggregate attribute lives on a non-root relation R_i . Here A is not visible at the root, and no formula above applies as stated. We instead carry a second annotation alongside μ : on the edge that lifts R_i towards the root, the same sorted group-by pass that emits $\sigma_j(v)$ also emits $\sigma_j^A(v) = \sum_{t_j: \pi_{K_{ij}}(t_j)=v} \mu(t_j) t_j[A]$, and each parent tuple combines the two by the product rule, so what travels up the tree is a per-key partial aggregate rather than individual A -values. At the root the aggregate is read off from these partials exactly as in Case 1.

REMARK 2. *Join-aggregate query verification is supported by combining the One-Pass OBJ with the aggregation above, at a total circuit cost of $O(IN + OUT)$.*

A worked example is provided in Appendix C, the cost breakdown in Appendix G.4. And we put the projection and predicates over the clean relations in Appendix E.

6.5 Join Results Enumeration

For a plain join, the prover enumerates the final output directly from the clean relations. Occurrences are named through the indexed relations $\hat{R}_i$ of Section 6.3, whose entries $(\ell, t_{\ell}, c(t_{\ell}))$ carry the committed row index and the clean indicator. Let $O = (o_1, \dots, o_M)$ denote the enumerated rows, for each row o_j , the prover supplies as auxiliary witness its *occurrence vector* $v_j = ((idx_{j,1}, t_{j,1}), \dots, (idx_{j,k}, t_{j,k}))$, where $t_{j,i}$ is the clean tuple of R_i^c contributing to o_j and $idx_{j,i}$ is its index ℓ in $\hat{R}_i$. Carrying the index rather than the tuple value alone is what keeps duplicate tuples apart, so two equal clean tuples contribute their own copies of the join result. The target is the multiset equality $O \equiv R_1^c \bowtie \dots \bowtie R_k^c$, certified without recomputing the join (Appendix E covers projection and cross-relation predicates). The circuit instead establishes the equality with four checks, each reusing a gate introduced earlier:

- (1) *Occurrence binding.* Each relation is presented as the index-augmented table $\hat{R}_i$ of Section 6.3, compressed to one field element per entry as in the semijoin gate (Section 6.1); a lookup argument checks the witness column $\{(idx_{j,i}, t_{j,i}, 1)\}_j$ against it.
- (2) *Join-row assembly.* For every tree edge (R_a, R_b) with shared attributes K_{ab} , an equality constraint per output row enforces $t_{j,a}[K_{ab}] = t_{j,b}[K_{ab}]$; by the running-intersection property of the join tree, pairwise agreement on all tree edges implies that the k referenced tuples assemble into a single consistent join tuple. The output columns of o_j are then wired to the attributes of the referenced tuples, so the claimed row is exactly the assembled tuple. Together, these rule out spurious and misassembled rows.
- (3) *Canonical ordering.* A linear gate packs each index vector into $\text{enc}(v_j) := \sum_{i=1}^k idx_{j,i} \cdot \rho^{k-i}$, the vector read as a k -digit number in a public radix $\rho > \max_i |R_i|$, so enc is injective; strict increase of consecutive encodings is enforced by range-checking $\text{enc}(v_{j+1}) - \text{enc}(v_j) - 1 \geq 0$ with the sort gate.
- (4) *Cardinality binding.* The multiplicity propagation of Section 6.3, run over the clean instance, gives the root sum $\sum_t \mu(t) = |R_1^c \bowtie \dots \bowtie R_k^c|$, and one equality constraint compares it with the claimed cardinality M , so no valid row can be omitted.

REMARK 3. *Plain join query verification is supported by combining the One-Pass OBJ with the enumeration above, at a total circuit cost of $O(IN + OUT)$.*

A worked example is given in Appendix D; we defer the complexity analysis to Appendix G.5.

7 TDJ: Efficient Verification for Cyclic Joins

The previous section developed the OBJ framework for acyclic queries, achieving $O(IN + OUT)$ circuit complexity with full obliviousness. However, many fundamental queries are inherently cyclic, such as cycle detection in graph analytics. Directly applying OBJ does not work, since cyclic queries admit no join tree to run it on (Section 3.1). In this section, we introduce the Tree-Decomposed Join Gate (TDJ), which reformulates the join structure into a tree whose nodes (*clusters*) absorb the cyclic joins. Once each intra-cluster join is materialized, the inter-cluster structure is acyclic and OBJ applies directly on top of it. The intra-cluster materializations are the only data-dependent piece. TDJ pads each of them to its worst-case intra-cluster bound, so the construction is *fully oblivious*. When full obliviousness is not required, one can go a step further and apply DP on top of TDJ, setting each intra-cluster capacity to a *DP-guided* upper bound instead of the worst-case bound. Because this bound sits strictly above the true cardinality while leaking only an (ϵ, δ) -DP amount of information about it, it relaxes full obliviousness to a quantifiable privacy budget on the released capacities while keeping the join result correct. We defer the formal (ϵ, δ) -obliviousness framework that unifies full obliviousness with this DP-based relaxation to Appendix H.1.

7.1 Tree-Decomposed Join Gate

A natural observation is that many cyclic queries can be re-structured as an acyclic join of materialized sub-joins, in which case OBJ can be applied directly on the outer acyclic structure once each sub-join has been obtained. For example, the triangle query $R_1(A, B) \bowtie R_2(B, C) \bowtie$

$R_3(C,A)$, whose join graph is a 3-cycle, can be re-expressed as a two-way join of the pre-materialized sub-join $R_1 \bowtie R_2$ (a single relation over (A,B,C)) with R_3 on keys (A,C) ; the outer two-relation structure is a tree, so OBJ handles it directly.

Building on this observation, we decompose the query hypergraph into a tree of clusters, where each cluster groups a subset of relations whose intra-cluster join the prover materializes, and the inter-cluster structure is a tree so OBJ applies at that level. The circuit verifies each intra-cluster join using standard binary join gates from [26]. This decomposition alone already yields a substantial complexity reduction: even with worst-case padding inside each cluster, the overall circuit size drops from $O(IN^k)$ to $O(IN^w)$, where w is the treewidth of the decomposition and is typically far smaller than k . Because the intra-cluster materialization sizes are data-dependent, however, worst-case padding can still be expensive. When full obliviousness is not required, one can optionally apply DP on top of TDJ, setting each intra-cluster capacity to a *DP-guided* upper bound on the true cardinality, which provides (ϵ, δ) -DP on the released capacity (and hence on the underlying intermediate cardinality) while sitting strictly above the true size so that no valid tuple is dropped. We expose this capacity as an abstract parameter in the gate below and defer the noise mechanism and its formal privacy analysis to Appendix H.2. The gate is expressed in the same style as OBJ:

DEFINITION 3 (TREE-DECOMPOSED JOIN GATE (TDJ)). *Fix a cyclic join query together with a tree decomposition into clusters $\{B_1, \dots, B_p\}$. We define the Tree-Decomposed Join Gate through the following three factors:*

- **Private witness.** *The input relations, the prover-materialized intra-cluster joins (with each column's capacity set to its worst-case intra-cluster bound for full obliviousness), padding/dummy indicators for those joins, and the per-relation indexed partitions used by the inter-cluster OBJ verification.*
- **Output.** *A certified clean representation of the cyclic join, equivalently the clean relations or cluster-relations from which the final query result is derived. As with OBJ, this output is consumed within the circuit by downstream operators.*
- **Verification target.** *The gate certifies that (i) every intra-cluster materialized relation is a correct join of the relations assigned to its cluster, and (ii) the resulting cluster relations satisfy the OBJ-style correctness conditions on the induced acyclic cluster tree.*

Realization via cyclic-to-acyclic reduction. To realize this reduction, we adopt the classical technique of tree decomposition [23, 28, 39], which has been widely used in database query processing and worst-case-optimal join algorithms precisely because it confines the combinatorial blowup of a cyclic query to within small local clusters, bounded by the treewidth parameter. Concretely, the query hypergraph is decomposed into a tree of clusters, where each cluster contains a subset of relations whose joins are materialized. Once the cluster tree is obtained, the One-Pass OBJ operates between clusters exactly as in the acyclic case: each cluster acts as a single relation whose tuples are the materialized intra-cluster join results.

Within each cluster, the prover materializes the join of the relations assigned to the cluster before it can participate in the inter-cluster OBJ. For a cluster containing relations $\{R_{i_1}, \dots, R_{i_w}\}$, the prover computes the join $R_{i_1} \bowtie \dots \bowtie R_{i_w}$ and provides the result as a witness column. The

circuit verifies this intra-cluster join using standard lookup-based binary join gates [26], which check that each output tuple has matching keys in all participating relations.

If a cluster contains w relations, then its worst-case intermediate size is $O(IN^w)$. Since the treewidth is the minimum width among all tree decompositions of the join hypergraph, a decomposition of width w localizes this blowup to individual clusters rather than the full query. As a result, the worst-case complexity drops from $O(IN^k)$ for the original k -way join to $O(IN^w)$, where w , the treewidth, is typically much smaller than k . In TDJ, each intra-cluster intermediate is padded to this worst-case bound, which hides the true cardinality completely and keeps the construction fully oblivious. *Optionally* replacing the worst-case bound with a DP-guided bound that is (i) strictly above the true cardinality (so correctness is preserved) and (ii) (ϵ, δ) -DP on the released capacity (so the underlying cardinality remains formally private) is precisely what Appendix H formalizes and instantiates, via the (ϵ, δ) -obliviousness spectrum (Appendix H.1) and the one-sided noise mechanism for cyclic joins (Appendix H.2).

8 Experiments

We evaluate VPJoin against PoneglyphDB [26] on both acyclic and cyclic multi-way join queries over TPC-H and graph workloads. Our system is built on Halo2 [50], a state-of-the-art ZK proof system. We compare against PoneglyphDB, the state-of-the-art ZK system for query answering, which uses the same Halo2 backend as VPJoin with identical security parameters (Pasta curves and approximately 128-bit security), so any performance gap can be attributed directly to the multi-way join verification strategy. Floating-point values are converted to 64-bit integers before circuit encoding, following prior work [26, 32]. All circuits are implemented in Rust using Halo2 [50].

8.1 Setup

Datasets. We use TPC-H [44] at three scales—60K, 120K, and 240K rows in the `lineitem` table. The default is 60K. For Q9, following prior work [32], we exclude `LIKE` predicates, as string operations are orthogonal to join verification. In addition, we use three SNAP [31] network datasets. The datasets are LastFM [2], which contains 7,624 nodes and 27,806 edges; Facebook [1], which contains 4,039 nodes and 88,234 edges; and Wikipedia Vote [3], which contains 7,115 nodes and 103,689 edges.

Queries. We select five TPC-H queries (Q3, Q5, Q8, Q9, Q18), spanning 3- to 8-way joins, and four graph queries (GQ1–GQ4) that we design ourselves over the three network datasets. GQ1 and GQ2 are 3- and 4-way path queries, respectively, while GQ3 and GQ4 are 3- and 4-way cycle queries, respectively. For cyclic queries, we further apply tree decomposition before running TDJ. In particular, the tree decompositions used for Q5, GQ3, and GQ4 are described in Appendix I.2.

Hardware and Runtime Environment. All experiments were conducted on a Linux server equipped with dual AMD EPYC 9555 64-core processors (128 cores / 256 threads) and approximately 2 TB of DDR5 memory. For both evaluated methods, witness generation was executed single-threadedly, while Halo2 proof construction was parallelized across CPU threads. Unless otherwise stated, we used the same thread configuration for both methods.

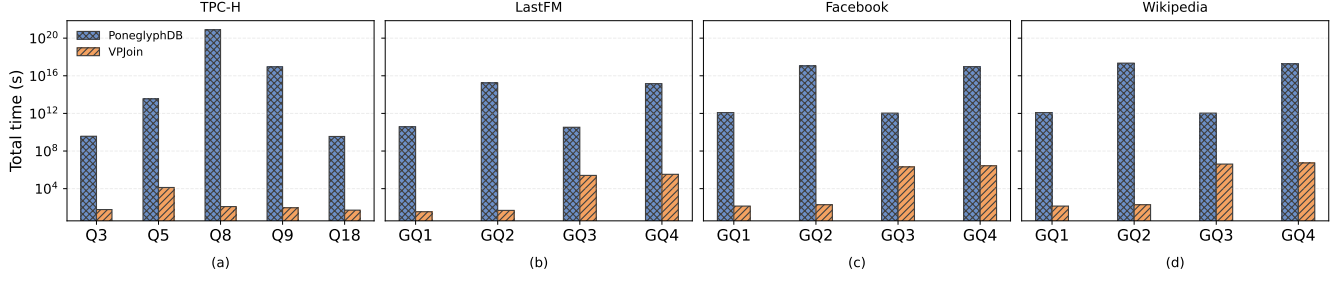


Figure 3: Proving time for 5 TPC-H SQL queries on the TPC-H benchmark and 4 graph SQL queries on 3 network datasets. All PoneglyphDB times are extrapolated estimates obtained via the estimation methodology; the VPJoin times for the fully oblivious cyclic queries GQ3 and GQ4 use the same extrapolation, from the anchors of Appendix J.

Parameter Setting. PoneglyphDB’s proving times, as reported in [26], can be improved by exploiting primary-key/foreign-key (PK–FK) constraints, that is, prior knowledge of the TPC-H schema, to bound the size of each intermediate join result. However, our design target is a method that works without any schema assumptions beyond the public relation sizes: many real deployments do not expose trustworthy constraints, and workloads such as the graph self-join queries in our experiments have no PK–FK structure at all. Assuming that PK–FK constraints are known to the verifier also corresponds to a simpler setting from the privacy standpoint, since such constraints reveal important structural properties of the joins and may partially expose join selectivities; not relying on them is standard in privacy-preserving query answering [29, 46]. For a fair head-to-head comparison, both systems here therefore use only public-size bounds, so VPJoin’s advantage comes from eliminating intermediates rather than from tighter bounds on them.

For cyclic queries, VPJoin optionally applies DP-guided padding (Appendix H.2), denoted VPJoin + DP. Unless the DP mode is explicitly indicated, every cyclic-query result labeled “VPJoin”, in particular all cyclic VPJoin bars in Figure 3, uses TDJ’s fully oblivious mode with worst-case intra-cluster padding. Unless otherwise stated, the overall privacy budget is $(\epsilon, \delta) = (0.1, 10^{-5})$, split equally across the intra-cluster estimates via basic composition.

Estimation Methodology for PoneglyphDB. Since the fully padded PoneglyphDB-style circuits cannot be run to completion even at the default 60K scale, the PoneglyphDB times in Figure 3 are *extrapolated from circuit size*. Following PoneglyphDB’s binary-join execution strategy under public-size bounds, we write Π_Q for the capacity that full obliviousness forces on the tallest materialized intermediate, i.e., the product of the public sizes of the relations combined into it, so the padded domain is $G_C = 2^{\lceil \log_2 \Pi_Q \rceil}$. We then scale a *measured* anchor to that domain, giving $T_{\text{est}} = T_0 \cdot G_C / G_{C,0}$: the anchor’s measured per-domain-row rate, charged for every row of the padded domain. The anchors are real, verified proofs on the same server rather than a cost model, and each is sized to fill its domain, so its rate is the one a fully padded circuit pays. TPC-H anchors on the released PoneglyphDB artifact; since that artifact provides no graph circuits, GQ1–GQ4 anchor on VPJoin’s own circuit for the same query. Proving time is near-linear in the domain size (Appendix A), and what the linear model omits runs in PoneglyphDB’s favor, so the reported times are floors. Appendix J gives the gate-row derivation, the measured anchors (Table 1), and the conservativeness analysis.

The same methodology yields the fully oblivious VPJoin times for GQ3 and GQ4, scaled from the anchors to the padded domains that TDJ’s worst-case intra-cluster capacities force (Appendix J).

Our implementation builds on the released PoneglyphDB codebase and inherits two simplifications of its benchmarking harness: the scaling experiments scale only the `lineitem` table, with the dimension tables kept at their base sizes, and the public input of every proof is trivial, with no externally published commitment $\text{Commit}(D)$ bound to it. In Appendix K, we report the additional per-query cost of binding each proof to the full database commitment $\text{Commit}(D)$.

8.2 Proving Time

We compare the end-to-end proving time of VPJoin and PoneglyphDB on both TPC-H and graph query workloads. All results are plotted on a logarithmic scale due to the vast difference in magnitude.

TPC-H Queries. Figure 3 (a) reports the proving time for the five TPC-H queries on the default 60K-row database. VPJoin achieves proving times on the order of 10^2 seconds for the four acyclic queries and roughly 1.3×10^4 seconds for the cyclic Q5. In contrast, PoneglyphDB’s estimated proving times range from approximately 3.5×10^9 seconds for the 3-way joins (Q3, Q18) to over 10^{20} seconds for Q8, the most complex 8-way join. The gap grows with join arity: for the 3-way joins, VPJoin is roughly 8 orders of magnitude faster; for Q5 and Q9 (both 6-way joins), the advantage is about 9 and 15 orders of magnitude; and for Q8 (an 8-way join), VPJoin is over 18 orders of magnitude faster.

This dramatic disparity stems directly from the difference in join verification strategy. PoneglyphDB decomposes each multi-way join into a sequence of binary joins and must provision circuit capacity for each materialized intermediate result, padded to worst-case cardinality bounds. For an 8-way join like Q8, the accumulated intermediate blowup renders the circuit large. VPJoin, by contrast, does the verification without materializing any intermediate join result. Among the five queries, the proving time of VPJoin on Q5 stands out because it is the only cyclic TPC-H query (join arity 6) for which VPJoin applies tree decomposition and pads intra-cluster intermediates to worst-case bounds (full obliviousness), resulting in a higher proving time ($\sim 10^4$ s) compared to the acyclic queries ($\sim 10^2$ s). Section 8.4 shows that DP-guided padding (Appendix H.2) reduces this overhead dramatically.

Graph Queries. Figure 3 (b)–(d) report the proving time for GQ1–GQ4 across the three network datasets. For the acyclic path queries (GQ1 and GQ2), VPJoin achieves proving times of approximately 10^2

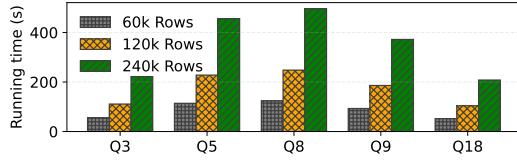


Figure 4: Proving time over data of increasing sizes. Q5 (cyclic) uses DP-guided padding with $\epsilon = 0.1$; all other queries use full obliviousness.

R2.03

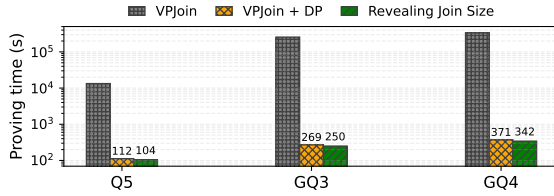


Figure 5: Impact of privacy constraints on proving time. VPJoin times (GQ3, GQ4) are extrapolated estimates (Section 8.1).

seconds, while PoneglyphDB’s estimated times range from 4×10^{10} seconds for the 3-way GQ1 to over 10^{17} seconds for the 4-way GQ2, yielding a gap of 9–15 orders of magnitude. For the cyclic queries (GQ3 and GQ4), VPJoin with full obliviousness (worst-case padding) reaches estimated times of 2.6×10^5 to 5.4×10^6 seconds, while PoneglyphDB’s times reach 3×10^{10} – 2×10^{17} seconds, giving VPJoin a clear 5–10 order-of-magnitude advantage.

Across all three datasets, the relative performance pattern remains consistent: the gap grows with join arity on both families, and at equal arity it is widest for the acyclic queries (GQ1, GQ2), where VPJoin pads nothing at all, and narrower for the cyclic queries (GQ3, GQ4), where VPJoin itself pays worst-case padding on the intra-cluster intermediate. As the dataset size grows from LastFM to Wikipedia, PoneglyphDB’s estimated times rise more rapidly than VPJoin’s, reflecting the multiplicative overhead of intermediate-result materialization on larger inputs.

8.3 Scalability

We evaluate how VPJoin’s proving time scales with increasing database size. Figure 4 reports the proving time for all five TPC-H queries at three scales: 60K, 120K, and 240K rows in the `lineitem` table. Since Q5 is the only cyclic TPC-H query, its fully oblivious cost is dominated by worst-case intra-cluster padding (see Section 8.4); to make the scalability trend comparable, we report Q5 with DP-guided padding at $\epsilon = 0.1$.

Across all queries, doubling the input size approximately doubles the proving time. For example, Q3 increases from roughly 55 s (60K) to 111 s (120K) to 222 s (240K), and Q18 follows a similar trajectory from 52 s to 104 s to 208 s. This near-linear scaling is consistent with the theoretical OBJ complexity of $O(IN + OUT)$ for acyclic queries.

Q5 is at approximately 114 s/228 s/456 s, and Q8 at approximately 124 s/248 s/496 s. Notably, Q8 is an 8-way join yet its proving time stays within 10% of Q5 (a 6-way join) and within a factor of about two of Q3 (a 3-way join). This demonstrates a key advantage of our verification approach: the proving cost is decoupled from the number of joins, scaling only with the data volume.

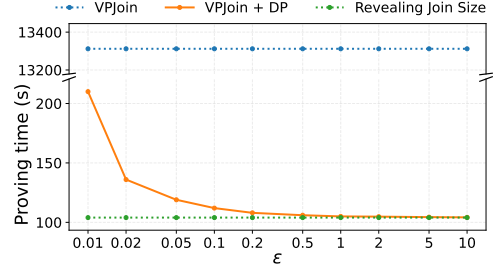


Figure 6: Proving time of Q5 for various ϵ values.

Scaling each of the dimension tables proportionally does not change the results above. We repeated the experiment with every table enlarged $2\times$ and $4\times$ alongside `lineitem`, and the proving time stays within 3% of the `lineitem`-only time at every scale. The per-table checks do grow with their tables, as the $O(IN + OUT)$ analysis predicts, but the circuit is charged for a padded power-of-two domain sized by its largest relation, `lineitem`, so the added rows fall inside capacity already allocated. It matters far more for PoneglyphDB, which pads each materialized intermediate to the product of its input sizes: doubling two joined tables quadruples it, and the factor compounds along the chain, so the reported gap would only widen. Appendix L reports the per-query numbers.

8.4 Privacy–Efficiency Trade-off for Cyclic Joins

For acyclic queries, VPJoin achieves full obliviousness at $O(IN + OUT)$ cost. Cyclic queries, however, require tree decomposition, which introduces intra-cluster intermediates. Padding to these worst-case bounds preserves full obliviousness but incurs substantial proving overhead. We therefore evaluate the DP-guided padding strategy (Appendix H.2), which replaces worst-case bounds with DP noisy cardinality estimates, and compare three privacy regimes:

- **VPJoin:** circuit capacities are set to worst-case upper bounds, revealing no information about intermediate cardinalities.
- **VPJoin + DP:** circuit capacities are set using noisy cardinality bounds satisfying (ϵ, δ) -DP, with controlled leakage.
- **Revealing Join Size:** circuit capacities match the true intermediate cardinalities, providing no privacy protection on join sizes but serving as a lower bound on proving time.

Overall comparison of the three regimes. Figure 5 reports the proving time of Q5 and the LastFM instances of GQ3 and GQ4 under each regime. Full obliviousness (VPJoin) incurs proving times of approximately 1.3×10^4 s for Q5 and 2.6×10^5 and 3.4×10^5 s for GQ3 and GQ4, reflecting the cost of worst-case padding on decomposed intermediates. In contrast, VPJoin + DP ($\epsilon = 0.1$) achieves speedups of roughly 2, 3, and 3 orders of magnitude, respectively. These DP-guided times are close to the Revealing Join Size baseline, incurring at most 10% overhead.

Various privacy budget ϵ . Figure 6 shows VPJoin + DP proving time as a function of ϵ for Q5, with the fully oblivious and Revealing Join Size baselines as reference lines. The DP curve decreases smoothly from approximately 210 s at $\epsilon = 0.01$ to 104 s at $\epsilon \geq 2$, converging to the no-privacy baseline. At the default $\epsilon = 0.1$, the proving time is approximately 112 s, only about 8% above the lower bound. The graph queries GQ3 and GQ4 exhibit the same trend: the DP-guided

proving time decreases monotonically as ϵ grows and converges to the Revealing Join Size baseline once ϵ reaches 1–2, at which point the added noise becomes negligible. Per-dataset breakdowns are provided in Appendix M.

References

- [1] 2024. SNAP: Facebook Ego-Network Dataset. <https://snap.stanford.edu/data/ego-Facebook.html>.
- [2] 2024. SNAP: LastFM Social Network Dataset. <https://snap.stanford.edu/data/feather-lastfm-social.html>.
- [3] 2024. SNAP: Wikipedia Vote Network Dataset. <https://snap.stanford.edu/data/wiki-Vote.html>.
- [4] Mahmoud Abo Khamis, Hung Q Ngo, and Atri Rudra. 2016. FAQ: Questions asked frequently. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 13–28.
- [5] Arvind Arasu, Ken Eguro, Raghav Kaushik, Donald Kossmann, Pingfan Meng, Vineet Pandey, and Ravi Ramamurthy. 2017. Concerto: A high concurrency key-value store with integrity. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 251–266.
- [6] Guillaume Bagan, Arnaud Durand, and Etienne Grandjean. 2007. On acyclic conjunctive queries and constant delay enumeration. In *Computer Science Logic: 21st International Workshop, CSL 2007*. Springer, 208–222.
- [7] Sumeet Bajaj and Radu Sion. 2013. CorrectDB: SQL engine with practical query authentication. *Proceedings of the VLDB Endowment* 6, 7 (2013), 529–540.
- [8] Stephanie Bayer and Jens Groth. 2012. Efficient zero-knowledge argument for correctness of a shuffle. In *EUROCRYPT*. 263–280.
- [9] Catriel Beeri, Ronald Fagin, David Maier, and Mihalios Yannakakis. 1983. On the desirability of acyclic database schemes. *J. ACM* 30, 3 (1983), 479–513.
- [10] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. 2016. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35*. Springer, 327–357.
- [11] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive* (2019).
- [12] Sean Bowe, Jack Grigg, and Daira Hopwood. 2020. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive*, Report 2019/1021. <https://eprint.iacr.org/2019/1021>.
- [13] Sean Bowe, Ying Tong Lai, Daira Hopwood, and Jack Grigg. [n.d.]. pasta_curves: implementation of the Pallas and Vesta (Pasta) curve cycle. https://github.com/zcash/pasta_curves. Accessed: 2026-04-18.
- [14] Wei Dong, Zijun Chen, Qiyao Luo, Elaine Shi, and Ke Yi. 2024. Continual observation of joins under differential privacy. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [15] Cynthia Dwork. 2006. Differential Privacy. In *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10–14, 2006, Proceedings, Part II (Lecture Notes in Computer Science)*, Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.). Springer, 1–12. https://doi.org/10.1007/11787006_1
- [16] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.
- [17] C. Christopher Erway, Alptekin Küpçü, Charalampos Papamanthou, and Roberto Tamassia. 2009. Dynamic Provable Data Possession. In *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*. 213–222.
- [18] Ariel Gabizon and Zachary J Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive* (2020).
- [19] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019).
- [20] Craig Gentry and Daniel Wichs. 2011. Separating Succinct Non-Interactive Arguments from All Falsifiable Assumptions. In *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC)*. 99–108.
- [21] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. 2015. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)* 62, 4 (2015), 1–64.
- [22] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. 1985. The Knowledge Complexity of Interactive Proof-Systems (Extended Abstract). In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6–8, 1985, Providence, Rhode Island, USA*, Robert Sedgewick (Ed.). ACM, 291–304. <https://doi.org/10.1145/22145.22178>
- [23] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 1999. Hypertree decompositions and tractable queries. In *Proceedings of the eighteenth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 21–32.
- [24] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 305–326.
- [25] Binbin Gu. 2025. *Integrating Zero-Knowledge Proofs for Efficient and Verifiable Data Systems*. Ph.D. Dissertation. University of California, Irvine.
- [26] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.

- [27] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive* (2022).
- [28] Manas R Joglekar, Rohan Puttagunta, and Christopher Ré. 2016. Ajar: Aggregations and joins over annotated relations. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 91–106.
- [29] Noah Johnson, Joseph P Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539.
- [30] Joe Kilian. 1992. A Note on Efficient Zero-Knowledge Proofs and Arguments (Extended Abstract). In *Proceedings of the 24th Annual ACM Symposium on Theory of Computing (STOC)*. 723–732.
- [31] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <https://snap.stanford.edu/data>.
- [32] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1804–1816.
- [33] Silvio Micali. 2000. Computationally Sound Proofs. *SIAM J. Comput.* 30, 4 (2000), 1253–1298.
- [34] Payman Mohassel and Saeed Sadeghian. 2013. How to hide circuits in MPC an efficient framework for private function evaluation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 557–574.
- [35] C Andrew Neff. 2001. A verifiable secret shuffle and its application to e-voting. In *CCS*. 116–125.
- [36] Charalampos Papamanthou, Roberto Tamassia, and Nikos Triandopoulos. 2008. Authenticated Hash Tables. In *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS)*. 437–448.
- [37] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. 2016. Pinocchio: Nearly practical verifiable computation. *Commun. ACM* 59, 2 (2016), 103–112.
- [38] Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. 2019. Efficient circuit-based PSI with linear communication. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 122–153.
- [39] Neil Robertson and Paul D Seymour. 1986. Graph minors. II. Algorithmic aspects of tree-width. *Journal of Algorithms* 7, 3 (1986), 309–322.
- [40] Jacob T Schwartz. 1980. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM* 27, 4 (1980), 701–717.
- [41] Rohit Sinha and Mihai Christodorescu. 2018. Veritasdb: High throughput key-value store with integrity. *Cryptology ePrint Archive* (2018).
- [42] Roberto Tamassia. 2003. Authenticated data structures. In *Algorithms-ESA 2003: 11th Annual European Symposium, Budapest, Hungary, September 16-19, 2003. Proceedings 11*. Springer, 2–5.
- [43] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology - CRYPTO 2013*. 71–89.
- [44] Transaction Processing Council. 2023. TPC-H Benchmark. <http://www.tpc.org/tpch/>. Accessed: 2023-xx-xx.
- [45] Chenghong Wang, Lina Qiu, Johes Bater, and Yukui Luo. 2024. SPECIAL: Synopsis Assisted Secure Collaborative Analytics. *arXiv preprint arXiv:2404.18388* (2024).
- [46] Yilei Wang and Ke Yi. 2021. Secure yannakakis: Join-aggregate queries over private data. In *Proceedings of the 2021 International Conference on Management of Data*. 1969–1981.
- [47] Yin Yang, Dimitris Papadimas, Stavros Papadopoulos, and Panos Kalnis. 2009. Authenticated join processing in outsourced databases. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 5–18.
- [48] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *VLDB*, Vol. 81. 82–94.
- [49] Andrew Chi-Chih Yao. 1986. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*. IEEE, 162–167.
- [50] Zcash. [n.d.]. halo2. <https://github.com/zcash/halo2>
- [51] Zcash. [n.d.]. Halo2 Protocol. <https://zcash.github.io/halo2/design/protocol.html>
- [52] Ce Zhang, Cheng Xu, Haixin Wang, Jianliang Xu, and Byron Choi. 2021. Authenticated keyword search in scalable hybrid-storage blockchains. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 996–1007.
- [53] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 863–880.
- [54] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. A zero-knowledge version of vSQL. *Cryptology ePrint Archive* (2017).
- [55] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. 2015. IntegriDB: Verifiable SQL for outsourced databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1480–1491.
- [56] Qingji Zheng, Shouhuai Xu, and Giuseppe Ateniese. 2012. Efficient query integrity for outsourced dynamic databases. In *Proceedings of the 2012 ACM Workshop on Cloud computing security workshop*. 71–82.

	(private) columns			public column	
	A	B	I	O	polynomial constraints:
row 1:	1	12	0	30	row 1: $I_1 - O = 0$
row 2:	2	20	12		row 2: $B_1 + I_1 - I_2 = 0$
row 3:	3	8	22		row 3: $B_2 + I_2 - I_3 = 0$
row 4:			30		row 4: $B_3 + I_3 - I_4 = 0$

→ denotes a copy constraint. For each row in the matrix, the multivariate polynomials are evaluated to 0.

Figure 7: PLONKish circuits illustration for SQL query “SELECT SUM(B) FROM R”.

A Proof Generation and Verification

We instantiate the proof system using the Halo2 framework [50, 51], the same backend used by PoneglyphDB [26], and inherit its cryptographic guarantees. We choose Halo2 for three reasons. First, its PLONKish arithmetization [19] supports *custom gates* that let us encode complex join-verification logic into compact, application-specific constraints. Second, it provides *native lookup arguments* [18], which our design exploits heavily for range checks and set-membership proofs. Third, PoneglyphDB [26] has already designed efficient gate primitives for relational operations (range checks, sorting, etc.) on the same backend, which we directly reuse and extend.

Setup. Halo2 operates over the Pasta curve cycle [11, 13] (255-bit field, $\lambda=128$ -bit security) and uses the Inner Product Argument (IPA) [12] for polynomial commitments, providing a *transparent* setup with no trusted third party. Throughout, $\mathbb{F}$ denotes this 255-bit Pasta scalar field, over which all circuit values, witness columns, and constraints are defined, and $|\mathbb{F}| \approx 2^{255}$ is its size.

Proof Complexity. In this paper, we use circuit size to refer to the total number of enforced constraints (equivalently, the number of circuit rows up to constant factors in our Halo2 implementation). We analyze each gate by its constraint cost and obtain the total circuit size by summing the costs of all instantiated gates. Let n denote the size of a circuit. The Halo2 IPA-based proof system achieves the following asymptotic bounds:

- **Proof size:** $O(\log n)$ group elements, yielding proofs of a few tens of kilobytes in practice.
- **Verification time:** $O(\log n)$ group operations, completing in under one second regardless of query complexity.
- **Proving time:** $O(n \log n)$. The prover performs $O(n \log n)$ field operations for polynomial arithmetic (FFT/NTT) and $O(n)$ elliptic-curve group operations for multi-scalar multiplications (MSMs) used in polynomial commitments and the IPA opening proof [12, 19]. Group operations are orders of magnitude more expensive than field operations, so the two terms carry comparable weight in practice [26] despite the differing asymptotics. The MSM term is moreover sub-linear in n at realistic sizes, because Pippenger’s window widens with the input length and the cost per point falls with it, while the FFT term is super-linear; the two deviations largely cancel, and the observed proving time scales almost linearly with n .

The sublinear proof size and verification time make the system *succinct*: the verifier’s cost is independent of the computation being proved. Since proving time scales nearly linearly with the circuit size, minimizing the number of constraints is the primary optimization objective, and more importantly, the central motivation for our witness-guided approach that avoids materializing intermediate join results.

Database commitment via IPA. The commitment $\text{Commit}(D)$ published during Setup (Section 4) is realized using the same IPA-based polynomial commitment scheme employed by the proof system itself, so no extra cryptographic primitive or assumption is introduced. Concretely, the prover canonicalizes the database $D = \{R_1, \dots, R_k\}$ into a single coefficient vector $\mathbf{d} = (d_0, \dots, d_{N-1}) \in \mathbb{F}^N$ over the Pasta scalar field (multi-attribute tuples are compressed to one field element, matching the convention used for lookup arguments in Section 5.2), and computes the Pedersen-style vector commitment

$$\text{Commit}(D) = \sum_{i=0}^{N-1} d_i G_i + rH,$$

where $\{G_i\}, H$ are independent Pasta-curve generators fixed by the transparent setup [13, 50] and $r \in \mathbb{F}$ is a uniformly random blinding scalar. This is a single multi-scalar multiplication of $N+1$ terms, so the prover’s commitment work is $O(N)$ while the output is a single group element ($O(1)$ space). Binding follows from the discrete-logarithm hardness of the Pasta curve, and hiding follows from the uniformity of r [10, 12].

Our implementation applies this scheme column-wise rather than to one concatenated vector: $\text{Commit}(D)$ is the vector of per-column commitments, one group element and one independent blinder per attribute column of D , so a query can open exactly the columns it reads instead of the whole database (Appendix K). The prover’s work is unchanged at $O(N)$ in total, the published size is one group element per column and therefore a function of the schema alone, and binding and hiding are the same argument applied per column.

Commitment verification. At query time, every subsequent ZK-SNARK proof is generated with respect to $\text{Commit}(D)$: the in-circuit witness values for the input relations are tied back to $\mathbf{d}$ via standard IPA *opening proofs* [10, 12, 19], which certify claimed evaluations of $f_D(X) = \sum_i d_i X^i$ at verifier-chosen challenge points. Each opening proof has size $O(\log N)$ group elements and is checked by the verifier in $O(\log N)$ group operations via the standard IPA reduction [12]; we reuse the protocol verbatim and inherit its security analysis. Because $\text{Commit}(D)$ is fixed before any query is issued, the binding property ensures every accepted proof is computed over the *same* database: to switch to any $D' \neq D$ that still opens to $\text{Commit}(D)$, the prover would have to produce a second pre-image of a Pedersen commitment, which is infeasible under discrete-log hardness. This is what discharges Threat (i) of the threat model (Section 4): any attempt by the prover to switch to a different database silently is caught at verification.

B PLONKish Circuit Example

Example B.1. Figure 7 illustrates a PLONKish circuit that evaluates the SQL query `SELECT SUM(B) FROM R`. The relation R consists of two attributes, A and B . The circuit uses two advice columns, A and B , to store the input tuples.

To perform the summation, we introduce an additional advice column I that stores intermediate partial sums. We denote the value in column $X \in \{A, B, I, O\}$ at row i (1-indexed) by X_i .

In row 1, we initialize the accumulator by setting $I_1 = 0$ and enforce the polynomial constraint $I_1 - 0 = 0$, which ensures that the summation starts from zero. For each subsequent row $i \geq 2$, the circuit adds the previous value in column B to the running sum. Specifically, in row 2 we add B_1 to I_1 and store the result in I_2 . This is enforced by the constraint $B_1 + I_1 - I_2 = 0$. Similarly, in rows 3 and 4, the circuit enforces $B_2 + I_2 - I_3 = 0$ and $B_3 + I_3 - I_4 = 0$, respectively, thereby accumulating the sum of all values in column B .

Finally, the value I_4 , which equals $\sum_{i=1}^3 B_i$, is copied into the public instance column at position O_1 using a copy (equality) constraint. This allows the verifier to read the final output of the computation while all intermediate values remain private.

C Aggregation Example

Example C.1 (Aggregation via tuple multiplicities). Continuing from Example 6.1 with the same clean relations $R_1^c = \{(1,1), (2,1)\}$, $R_2^c = \{(1,11)\}$, $R_3^c = \{(11,4), (11,2)\}$ and join tree rooted at R_2 , we compute `SELECT A, COUNT(*), SUM(D) ... GROUP BY A` without materializing the join result (see Figure 8).

- (1) **Initialize leaf multiplicities.** Every tuple in a leaf relation gets $\mu = 1$: $R_1^c: (1,1) \mapsto 1, (2,1) \mapsto 1$; $R_3^c: (11,4) \mapsto 1, (11,2) \mapsto 1$.
- (2) **Process edge (R_2, R_1) on B .** Sort R_1^c by B and group: key $B=1$ has $\Sigma\mu = 1+1=2$. Lookup into R_2^c : tuple $(1,11)$ receives $\mu_L = 2$.
- (3) **Process edge (R_2, R_3) on C .** Sort R_3^c by C and group: key $C=11$ has $\Sigma\mu = 1+1=2$. In the same pass, compute the partial aggregate $\Sigma\mu \cdot D = 1 \cdot 4 + 1 \cdot 2 = 6$ for key $C=11$. Lookup into R_2^c : tuple $(1,11)$ receives $\mu_R = 2$ and partial $\text{SUM}(D) = 6$.
- (4) **Multiply at root.** $\mu(1,11) = \mu_L \times \mu_R = 2 \times 2 = 4$.
- (5) **Group-by and final aggregates.** Since the query groups by A and $A \in R_1$ (a leaf), per-group aggregates are computed at R_1^c . Each tuple $(a,1) \in R_1^c$ matches the root tuple $(1,11)$, whose right-branch multiplicity is $\mu_R = 2$ and partial $\text{SUM}(D) = 4 + 2 = 6$.
 - $A=1$: $\text{COUNT}(\ast) = 1 \cdot \mu_R = 2$, $\text{SUM}(D) = 1 \cdot 6 = 6$.
 - $A=2$: $\text{COUNT}(\ast) = 1 \cdot \mu_R = 2$, $\text{SUM}(D) = 1 \cdot 6 = 6$. $\text{AVG}(D)$ per group is simply $\text{SUM}(D) / \text{COUNT}(\ast) = 6/2 = 3$.

These match the full join result $\{(1,1,11,4), (2,1,11,4), (1,1,11,2), (2,1,11,2)\}$ grouped by A : each group has 2 tuples with $D \in \{4,2\}$, giving $\text{COUNT} = 2$ and $\text{SUM}(D) = 6$ —all computed without materializing the join.

D Join Results Enumeration Example

Example D.1 (Join results enumeration via the four checks). Continuing from Example 6.1, consider the same clean relations

$$R_1^c = \{(1,1), (2,1)\}, \quad R_2^c = \{(1,11)\}, \quad R_3^c = \{(11,4), (11,2)\},$$

for the query

$$R_1(A, B) \bowtie R_2(B, C) \bowtie R_3(C, D),$$

with the join tree rooted at R_2 . The prover claims the following $M=4$ output rows over (A, B, C, D) :

$$\begin{aligned} o_1 &= (1, 1, 11, 4), & o_2 &= (1, 1, 11, 2), \\ o_3 &= (2, 1, 11, 4), & o_4 &= (2, 1, 11, 2). \end{aligned}$$

The partition is that of Example 6.2, and indices are inherited from the committed relations

$$\begin{aligned} R_1 &= \{(1,1), (2,1), (3,2), (4,3), (5,6)\}, \\ R_2 &= \{(1,11), (2,12), (3,13), (4,14), (5,16)\}, \\ R_3 &= \{(10,1), (11,4), (11,2), (15,3), (16,5)\}, \end{aligned}$$

so the occurrences carrying $c=1$ are

$$\begin{aligned} R_1: 1 &\mapsto (1,1), & 2 &\mapsto (2,1), \\ R_2: 1 &\mapsto (1,11), \\ R_3: 2 &\mapsto (11,4), & 3 &\mapsto (11,2), \end{aligned}$$

the remaining occurrences carrying $c=0$. The indices distinguish tuple occurrences rather than only tuple values. Thus, if a clean relation contained two equal tuples at different indices, both occurrences could contribute separate copies to the join result under multiset semantics.

For each claimed row o_j , the prover supplies an occurrence vector

$$v_j = ((idx_{j,1}, t_{j,1}), (idx_{j,2}, t_{j,2}), (idx_{j,3}, t_{j,3})),$$

where $(idx_{j,i}, t_{j,i})$ identifies the selected row of R_i used to assemble o_j . In this example, the index components are

$$(1,1,2), \quad (1,1,3), \quad (2,1,2), \quad (2,1,3).$$

Using the public radix $\rho = 10 > \max_i |R_i| = 5$, the circuit encodes them as

$$\begin{aligned} \text{enc}(v_1) &= 1 \cdot \rho^2 + 1 \cdot \rho + 2 = 112, \\ \text{enc}(v_2) &= 1 \cdot \rho^2 + 1 \cdot \rho + 3 = 113, \\ \text{enc}(v_3) &= 2 \cdot \rho^2 + 1 \cdot \rho + 2 = 212, \\ \text{enc}(v_4) &= 2 \cdot \rho^2 + 1 \cdot \rho + 3 = 213. \end{aligned}$$

The circuit then applies the four checks of Section 6.5.

- (1) **Occurrence binding.** The index-augmented table of each relation carries the committed row index and the clean indicator. For example,

$$\widehat{R}_3 = \{(1, (10,1), 0), (2, (11,4), 1), (3, (11,2), 1), (4, (15,3), 0), (5, (16,5), 0)\}.$$

For o_3 , the supplied occurrence vector refers to

$$(2, (2,1), 1) \in \widehat{R}_1, \quad (1, (1,11), 1) \in \widehat{R}_2, \quad (2, (11,4), 1) \in \widehat{R}_3.$$

The circuit verifies all three references by lookup, each with the indicator component fixed to 1. Because a lookup binds the index, the tuple value and the indicator together, the prover cannot fabricate a tuple, attach a genuine tuple to a false index, or reference a residual occurrence. For instance, $(2, (2,12), 1)$ fails because row 2 of R_2 carries $c=0$, and $(1, (9,99), 1)$ fails because row 1 of R_2 is $(1,11)$.

- (2) **Join-row assembly.** The circuit next checks that the referenced tuples form a valid join row. On edge (R_2, R_1) , the shared attribute is B , and on edge (R_2, R_3) , the shared attribute is C . For o_3 , the referenced tuples are

$$t_{3,1} = (2,1), \quad t_{3,2} = (1,11), \quad t_{3,3} = (11,4).$$

They satisfy

$$t_{3,1}[B] = 1 = t_{3,2}[B], \quad t_{3,2}[C] = 11 = t_{3,3}[C].$$

Agreement on all join-tree edges implies that the three tuples assemble into one globally consistent join tuple. The output cells are then wired to the referenced attributes:

$$A = t_{3,1}[A] = 2, \quad B = t_{3,2}[B] = 1, \quad C = t_{3,2}[C] = 11, \quad D = t_{3,3}[D] = 4.$$

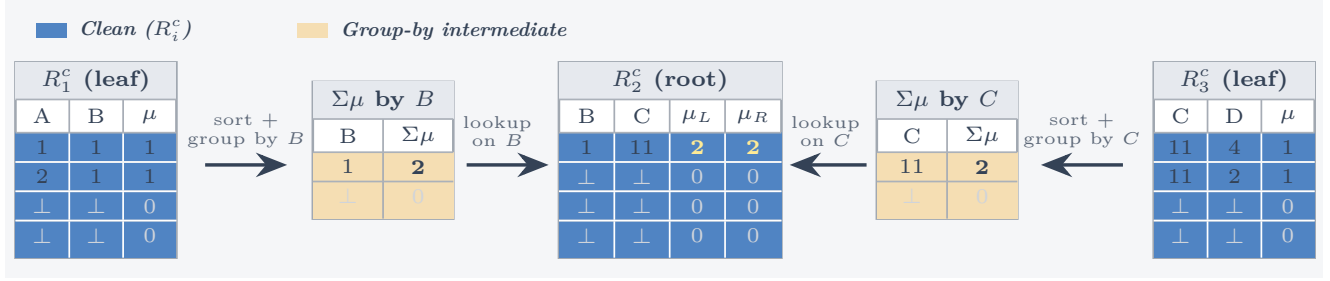


Figure 8: Aggregation via tuple multiplicities for `SELECT A, COUNT(*) , SUM(D) FROM R1, R2, R3 WHERE R1.B = R2.B AND R2.C = R3.C GROUP BY A`.

Hence the occurrence vector reproduces exactly $o_3 = (2, 1, 11, 4)$. Claiming $(2, 1, 11, 9)$ with the same occurrence vector would fail because $9 \neq t_{3,3}[D] = 4$. Likewise, selecting genuine tuples whose shared keys disagree would fail the edge equality constraints.

- (3) **Canonical ordering.** The circuit requires the occurrence-vector encodings to be strictly increasing:

$$112 < 113 < 212 < 213.$$

Concretely, it range-checks

$$\text{enc}(v_{j+1}) - \text{enc}(v_j) - 1 \geq 0,$$

giving

$$113 - 112 - 1 = 0, \quad 212 - 113 - 1 = 98, \quad 213 - 212 - 1 = 0.$$

All three checks pass. Because ρ is larger than every possible tuple index, enc is injective on index vectors. Strict ordering therefore guarantees that no tuple-occurrence combination is reported twice. For example, claiming o_2 a second time would create adjacent encodings 113, 113 and require

$$113 - 113 - 1 = -1 \geq 0,$$

which fails. Importantly, the ordering is imposed on occurrence vectors rather than output values, so two equal-valued output rows generated by different input occurrences remain valid under multiset semantics.

- (4) **Cardinality binding.** The multiplicity propagation of Example C.1 independently certifies the true number of join occurrences. The root tuple $(1, 11) \in R_2^c$ matches two tuples in R_1^c on $B=1$ and two tuples in R_3^c on $C=11$. Its multiplicity is therefore

$$\mu(1, 11) = (\mu(1, 1) + \mu(2, 1))(\mu(11, 4) + \mu(11, 2)) = (1+1)(1+1) = 4.$$

Since R_2^c contains only this root tuple and every other row of R_2 carries $\mu^c = 0$, the certified join cardinality is

$$\sum_{t \in R_2} \mu^c(t) = 4.$$

The circuit compares this value with the public output length $M=4$. Omitting o_4 and claiming only $M=3$ rows would therefore fail the equality check.

The first two checks establish that every claimed row is generated by a valid combination of genuine clean-tuple occurrences. Canonical ordering shows that these combinations are pairwise distinct, while cardinality binding shows that their number equals the total number of

valid join combinations. Therefore, none can be fabricated, duplicated, or omitted, and the claimed output satisfies the exact multiset equality

$$\{o_1, o_2, o_3, o_4\} \equiv R_1^c \bowtie R_2^c \bowtie R_3^c.$$

Thus, the claimed output matches Example 6.1, and each cheating strategy described in Section 6.5 is ruled out by its corresponding check.

E Projection and Predicates with Clean Relations

Projection. Many analytical queries project the join result onto a subset of attributes (e.g., `SELECT R1.A, R2.B FROM R1 JOIN R2 JOIN R3 . . .`). In existing ZK database systems, projection is applied only after the full join is materialized, so the circuit must still accommodate the entire $O(IN^k)$ intermediate. In our framework, the OBJ produces clean relations R^c that are subsets of the original input tables. When the query is a free-connex acyclic query—that is, the output attributes form a connected subset within the join tree—the final join result can be enumerated directly from the clean relations, and projection is applied during this enumeration by copying only the required attribute columns to the output; the claimed (projected) output is verified against the clean relations by the final join enumeration of Section 6.5. Relations outside the output subtree contribute only to the gate’s consistency and counting conditions (filtering non-participating tuples), but their non-key attributes never appear in the output circuit. Since projection merely selects a subset of columns from the clean relations and the enumerated output, it introduces no additional constraints beyond those already required by the One-Pass OBJ and the final join enumeration.

In-Relation Predicates. Queries frequently include selection predicates on a single relation—for example, `WHERE R1.price > 100`. Because each such predicate references only attributes within one input table, it is evaluated per tuple and folded into the clean/residual split rather than applied to a materialized intermediate: the prover places tuples that fail the condition in the residual part, and the circuit certifies that placement. For each tuple t of R_i the circuit certifies the bit $b(t)$ as the exact truth value of the predicate, by a booleanity gate together with the range check of Section 5.2 on the comparison, and enforces $c(t)(1 - b(t)) = 0$ against the clean indicator, which is the predicate requirement that the correctness argument of Appendix F.2 appeals to. The predicate is enforced on the clean side rather than on all of R_i : a real input contains failing tuples, so requiring p_i of every row of R_i would be unsatisfiable even for an honest prover. This adds at most

$O(|R_i|)$ constraints per predicated relation, fully absorbed into the existing $O(IN)$ bound. Both root sums in (9) count joins of filtered relations, the input channel being anchored at b at every tuple, so a tuple that fails the predicate contributes to neither side and its placement in the residual part costs nothing. Correctness then follows as in Appendix F.2, whose argument reads $\mathcal{R}^P$ for the input instance throughout.

Cross-Relation Predicates. Queries often include selection predicates that reference attributes from multiple relations—for example, `WHERE R1.price > R2.budget`. In existing systems, such predicates can only be evaluated after materializing the binary join that brings these attributes together, again requiring $O(IN^k)$ worst-case circuit capacity. In contrast, in our framework, cross-relation predicates are enforced within the final join enumeration of Section 6.5, which assembles each claimed output row from its clean tuples. For each output tuple—which combines attributes from multiple clean relations—the circuit enforces the predicate as an additional arithmetic constraint (e.g., a range check for inequality predicates or an equality gate for exact matches). The predicate selects within the enumeration rather than shortening it: every clean-join row is enumerated and carries a satisfaction bit, and the returned answer is the sub-multiset with the bit set, which is what keeps the cardinality binding of Section 6.5 exact. An aggregate is no different: a cross-relation predicate cannot be folded into the clean/residual split of any single relation, so the aggregate is taken over the same enumerated rows, weighted by their satisfaction bits, and its cost is the size of that enumeration rather than the number of groups. The graph queries of Appendix I.1 are of this form, since their path and cycle patterns carry vertex-ordering predicates across relations. Since these constraints are applied per output row, they add at most $O(OUT)$ constraints and do not affect the overall $O(IN + OUT)$ complexity. Importantly, no intermediate relation needs to be materialized to evaluate these predicates: the clean relations already identify which input tuples participate, and the predicate is checked only on the final combined output.

F Correctness and Obliviousness Analysis

F.1 Two-Pass OBJ

Correctness. To prove that $\{R_1^c, \dots, R_k^c\}$ contains exactly the tuples participating in the global join, we verify two properties. Note that these are properties of the *verified join result*, distinct from the proof-system-level soundness and completeness defined in Section 3.2.

(1) *No false positives (every retained tuple is valid).* For each semijoin gate on edge (P, C) , the Membership Check enforces $\pi_{K_{PC}}(R_P^c) \subseteq \pi_{K_{PC}}(R_C)$, so only tuples with matching keys survive. By induction on the join tree, the bottom-up and top-down passes implement the Yannakakis semijoin reduction, and the pairwise consistency check confirms key agreement on every edge. Thus every tuple in R_i^c is fully connected in the global join.

(2) *No false negatives (no valid tuple is discarded).* For each semijoin gate on edge (P, C) , the Conservation Check ensures $R_P \equiv R_P^c \cup R_P^r$ (no tuples lost), and the Non-Membership Check enforces $\pi_{K_{PC}}(R_P^r) \cap \pi_{K_{PC}}(R_C) = \emptyset$. If a matching tuple were placed in R_P^r , it would violate non-membership, a contradiction. Since each gate preserves all valid tuples and the two-pass reduction is exhaustive for acyclic queries, the final $\{R_1^c, \dots, R_k^c\}$ contains every tuple participating in the global join.

Obliviousness. The OBJ gate achieves obliviousness through data-independent circuit structuring; each relation occupies a column of

its own public size $|R_i|$, with no uniform worst-case padding across relations.

- *Structural Uniformity.* The number of advice columns, their lengths, and the polynomial constraints are determined solely by the query, the public schema, and the public relation sizes $\{|R_1|, \dots, |R_k|\}$. The wiring of the permutation and lookup arguments is fixed at compile time and does not branch on tuple values.
- *Data Hiding via Private Witness.* Tuple values are hidden by the zero-knowledge property of the underlying commitment scheme (IPA commitments in Halo2). The prover supplies the per-edge partition (R_i^c, R_i^r) as private witnesses, so the verifier sees only commitments to columns of public length $|R_i|$ without learning their contents.
- *Cardinality Hiding.* The split sizes $|R_i^c|$ and $|R_i^r|$ are data-dependent, but together they occupy exactly $|R_i|$ rows, a public quantity. Their individual values appear only as positions within these $|R_i|$ -row columns and are masked by the private witness, so the verifier learns nothing about join selectivity or intermediate join cardinalities.

F.2 One-Pass OBJ

Correctness. We prove that $\mathcal{R}^c$ is the fully reduced instance, i.e., $R_i^c \equiv R_i^*$ as multisets for every i , where $\mathcal{R}^* = \{R_1^*, \dots, R_k^*\}$ is the fully reduced instance of the predicate-filtered query, i.e., R_i^* holds the occurrences of R_i^P that extend to a full join row. Because relations are bags, the argument is carried out over tuple *occurrences* rather than tuple values: an occurrence is a row of an input relation, and an *occurrence combination* is a vector $(r_1, \dots, r_k)$ of rows, one per relation, agreeing on every join key and each satisfying its own in-relation predicate. Write Ω for the set of occurrence combinations of $\mathcal{R}^P$ and Ω^c for those all of whose rows lie in the clean parts. By (7) a clean occurrence is an occurrence of R_i , and by the *predicate requirement*, the constraint $c(t)(1 - b(t)) = 0$ that the circuit enforces on every row of a predicated relation (Appendix E), it satisfies its predicate, so it is an occurrence of R_i^P ; hence $\Omega^c \subseteq \Omega$. Because the indices in R_i are distinct, the single Conservation Check behind (7) already rules out an occurrence being fabricated, lost, duplicated, or counted on both sides, so no separate disjointness condition is needed. The check is also what makes the partition well defined. The prover supplies the two parts as separate witness columns whose entries carry the constant 1 and the constant 0 in the indicator position, so matching a committed row $(t, t_c, c(t_c))$ against one of them forces $c(t_c)$ to be boolean and to equal the tag of the part the row was placed in. Booleanity is not cosmetic here: c is multiplied into the counts below as an anchor, and an advice value outside $\{0, 1\}$ would scale a clean-side multiplicity and let an occurrence hidden in a residual part be compensated for. The check also fixes one partition for the whole gate, since (8), (9) and the enumeration of Section 6.5 all read the split through that same c column. As with the Two-Pass variant, the analysis concerns the verified join result, not the proof system itself: each condition is enforced by a permutation, lookup or sort argument that is sound up to the negligible error of Section 3.2, so the gate inherits that error by a union bound.

The two annotations that (9) compares are two instances of one mechanism. On the committed rows an instance is named by a boolean

indicator, its *anchor* a , and the propagation is run in the anchored form

$$\mu^a(t_i) = a(t_i) \prod_{R_j \text{ child of } R_i} \sigma_j^a(t_i), \quad \sigma_j^a(t_i) = \sum_{\substack{t_j \in R_j \\ \pi_{K_{ij}}(t_j) = \pi_{K_{ij}}(t_i)}} \mu^a(t_j),$$

with $a = c$ on the clean channel and $a = b$ on the input channel, $b \equiv 1$ on a relation carrying no predicate. The sums range over all of R_j and a row outside the instance carries $\mu^a = 0$, so every column keeps its committed height and none is shortened to the secret clean cardinality; this is the form in which the per-edge checks of Section 6.3, written there over the clean parts, are realized. The recursion is started by one equality per leaf row and channel, pinning $\mu^a(t) = a(t)$ at the leaves. That base case is what the induction below rests on: the per-edge checks fix each parent's annotation as a product of certified grouped sums but constrain the leaves not at all, and a prover free to inflate a leaf multiplicity could raise the clean root sum to cover an occurrence hidden in a residual part.

The two root sums that (9) compares are indeed $|\Omega^c|$ and $|\Omega|$: by the propagation of Section 6.3, whose per-edge checks certify each factor as the grouped sum of the child multiplicities over the merged key columns and whose anchors are multiplied in at every node, the annotations are determined bottom-up and neither a reweighting of the factors nor an inflation of a child multiplicity is available, so by induction $\mu^c(t) = c(t) \cdot n^c(t)$ and $\mu^{\text{all}}(t) = b(t) \cdot n^p(t)$, where $n^c(t)$ and $n^p(t)$ count the extensions of t into its subtree through clean and through predicate-satisfying rows respectively. Summing over the root relation gives $|\Omega^c|$ and $|\Omega|$. Since $\Omega^c \subseteq \Omega$, a subset of equal finite cardinality is the whole set, so (9) forces $\Omega^c = \Omega$. The comparison is an equality of field elements and is an equality of the two counts because both are at most $\prod_i |R_i|$, far below the modulus of the Pasta scalar field, so neither sum wraps.

(1) *No false positives (every clean tuple is valid)*. (8) enforces that for every edge (R_i, R_j) in the join tree, $\pi_{K_{ij}}(R_i^c) = \pi_{K_{ij}}(R_j^c)$. For an acyclic query on a join tree, pairwise consistency on every edge implies global consistency [9], so every tuple of every R_i^c extends to a full join row of $\mathcal{R}^c$, and by the predicate requirement that row is also a join row of $\mathcal{R}^p$. Participation depends only on the tuple value, and $R_i^c \preceq R_i^p$, so $R_i^c \preceq R_i^*$. (2) *No false negatives (no valid tuple is hidden)*. (9) gives $\Omega^c = \Omega$. Now let r be any row of R_i whose tuple lies in R_i^* . Then r appears in some combination $\tau \in \Omega$, hence $\tau \in \Omega^c$, and every row of a combination of Ω^c lies in the clean parts, so $r \in R_i^c$. Every participating occurrence is therefore clean, giving $R_i^* \preceq R_i^c$; with part (1) this proves $R_i^c \equiv R_i^*$.

Working over occurrences rather than values is what makes this hold under bag semantics. If R_i holds a participating tuple twice, each copy is its own indexed occurrence; hiding one removes exactly its combinations from Ω^c , and no rearrangement of the partition can compensate, since Ω^c never exceeds Ω . The value-level ambiguity that forces a separate disjointness condition on an unindexed split cannot arise.

The counting formulation is also what makes the argument work where a traversal of the residual side does not. A participating occurrence moved to the residual part may leave every one of its join partners clean, in which case it is invisible to any condition phrased over the residual rows alone; what it cannot do is leave the number of join occurrences unchanged.

Completeness. An honest prover places exactly the participating occurrences of the filtered query in the clean parts and satisfies all

three conditions: (7) holds because that assignment is a partition of the occurrences, and the predicate requirement because participating rows satisfy their predicates; (8) holds because the fully reduced instance is pairwise consistent; and (9) holds because the rows of $\mathcal{R}^p$ outside $\mathcal{R}^*$ take part in no occurrence combination, so placing them in the residual parts leaves Ω unchanged and $\mathcal{R}^c = \mathcal{R}^*$ gives $\Omega^c = \Omega$.

Obliviousness. The One-Pass OBJ maintains obliviousness under the same principles:

- For each node R_i , the prover supplies $\widehat{R}_i^c$ and $\widehat{R}_i^p$ as private witnesses. Since the partition is exhaustive, the two parts together occupy exactly $|R_i|$ rows, which is public; the split sizes are hidden within these rows and inaccessible to the verifier through the zero-knowledge commitments, so the verifier learns nothing about how many rows survive.
- The three conditions (7)–(9) are enforced by Permutation Arguments, gated Membership Checks, and the sorted pass and lookup of the Cardinality Preservation Check, whose circuit layouts depend only on the public sizes $\{|R_1|, \dots, |R_k|\}$; the fixed selectors that confine lookup expressions to the real rows are determined by the query and those sizes alone. The verifier's computational path is constant and independent of the clean/residual ratio, so the query selectivity is never leaked through the circuit.
- The additional witness data of (9), two multiplicities per row of $\mathcal{R}$, occupies advice columns of the same public height as the relations themselves and is hidden by the zero-knowledge property, exactly as the partition is. The two root sums are compared inside the circuit and neither is exposed, so (9) itself discloses nothing. What the query already reveals is unchanged by it: a plain join returns its rows, a COUNT(*) aggregate returns the join size by definition, and a grouped COUNT reveals it as the sum of its groups, whereas a SUM aggregate reveals neither.

F.3 Join Results Enumeration

Correctness. The target is $O \equiv J(\mathcal{R}^c)$, the join of the clean relations certified by either OBJ variant. Identify each occurrence of $J(\mathcal{R}^c)$ with the vector of row indices that generates it; distinct vectors give distinct occurrences even when the tuples coincide, which is what makes the target a multiset statement rather than a set one.

Occurrence binding forces every pair $(idx_{j,i}, t_{j,i})$ to name a genuine clean occurrence at its committed index, and join-row assembly forces the k named tuples to agree on every tree edge; by the running-intersection property they therefore form an occurrence of $J(\mathcal{R}^c)$, and o_j is wired to be that row. Every claimed row is thus an occurrence of $J(\mathcal{R}^c)$, which rules out spurious and misassembled rows. Canonical ordering makes the M index vectors pairwise distinct, so the claimed rows are M distinct occurrences and $O \preceq J(\mathcal{R}^c)$ with $M \leq |J(\mathcal{R}^c)|$. Cardinality binding gives $M = |J(\mathcal{R}^c)|$, and a sub-multiset of equal cardinality is the whole, so $O \equiv J(\mathcal{R}^c)$. Conversely an honest prover meets all four checks, so the enumeration is complete as well as sound.

Obliviousness. The occurrence vectors and the index-augmented tables are private witness columns hidden by the zero-knowledge property, and the circuit layout depends only on the public sizes $|R_1|, \dots, |R_k|$ and on the public length M of the enumeration; what the enumeration covers, and hence what M counts, is stated in Appendix E for queries carrying a cross-relation predicate.

G Detailed Complexity Analysis

This appendix gives the full constraint-cost derivation. Instead of padding every relation to a uniform capacity, we keep each relation at its actual public size $|R_i|$ and express every cost as a function of these sizes. All totals are then expressed in terms of the total input size $IN = \sum_{i=1}^k |R_i|$.

G.1 Primitive Costs

Let $B=64$ denote the key bit width and $b=8$ the range-check chunk size, giving $t = \lceil B/b \rceil = 8$ range-check chunks per value. Since every column length is at most IN , each primitive check runs in $O(IN)$ constraints; we spell out the per-column expression to expose the constant factors used later.

- **Conservation Check** (permutation argument verifying multi-set equality of two columns of the same length n): n constraints, hence $O(IN)$.
- **Membership Check** with witness column X and table column Y (Plookup): $\max(|X|, |Y|)$ constraints, hence $O(IN)$.
- **Non-Membership Check** on columns X and Y via merge-and-sort has two parts:
 - a permutation argument binding the prover-supplied sorted sequence to the merge of X and Y : $|X| + |Y|$ constraints;
 - a sortedness proof range-checking consecutive differences: $t(|X| + |Y|) = 8(|X| + |Y|)$ constraints.

Standalone, a Non-Membership Check thus costs $9(|X| + |Y|) = O(IN)$. When the gate already contains a Conservation Check (a permutation argument on the same rows), the Non-Membership permutation merges with it under PLONKish's global permutation argument, so only the $8(|X| + |Y|) = O(IN)$ sortedness cost is added on top.

G.2 Semijoin Gate Cost

A semijoin gate $S_{j \rightarrow j}$ verifying the partition of R_i against R_j invokes:

- (1) Conservation on R_i vs. $R'_i \cup R''_i$: $|R_i|$ constraints.
- (2) Membership $\pi_{K_{ij}}(R'_i) \subseteq \pi_{K_{ij}}(R_j)$: $\max(|R'_i|, |R_j|) \leq \max(|R_i|, |R_j|)$ constraints.
- (3) Non-Membership $\pi_{K_{ij}}(R''_i) \cap \pi_{K_{ij}}(R_j) = \emptyset$: because the gate already contains a Conservation permutation, only the sortedness cost is added: $8(|R'_i| + |R_j|) \leq 8(|R_i| + |R_j|)$ constraints.

Summing:

$$C(S_{j \rightarrow j}) \leq |R_i| + \max(|R_i|, |R_j|) + 8(|R_i| + |R_j|) = O(|R_i| + |R_j|). \quad (10)$$

G.3 Two-Pass OBJ Total

The Two-Pass OBJ applies one semijoin gate per edge in each of the bottom-up and top-down passes, for $2(k-1)$ gates total. Summing (10) over all edges:

$$C_{2\text{-Pass}} \leq 2 \sum_{(P,C) \in T} O(|R_P| + |R_C|) = O\left(\sum_{v \in T} \deg(v) |R_v|\right),$$

where $\deg(v)$ is the degree of node v in the join tree. Since $\sum_v \deg(v) = 2(k-1)$, we have $\sum_v \deg(v) |R_v| \leq d_{\max} IN$ where $d_{\max} = \max_v \deg(v)$. For a fixed query (constant k , constant tree shape):

$$C_{2\text{-Pass}} = O(IN).$$

G.4 Multiplicity Propagation and Aggregation Cost

The propagation of Section 6.3 traverses each tree edge once. Per edge (R_i, R_j) it sorts R_j by K_{ij} , at $|R_j|$ for the permutation binding plus $8|R_j|$ for the sortedness proof; scans the sorted rows to emit the per-key sums, at one boundary constraint and one running addition per row, i.e., $2|R_j|$; performs one lookup per parent row against the key-indexed table, $|R_i|$; and folds each result into a running product, $|R_i|$, plus one equality per row to seat that running product at the row's anchor, which at a leaf, having no child to fold in, is the whole of μ^a . The heights are the committed ones throughout, since the clean cardinality is secret and no column may be shortened to it. The three checks are written in Section 6.3 over the clean parts, as the Two-Pass OBJ materializes them; when the clean instance is given instead by the indicator on the committed rows, they run unchanged over those rows with the anchor multiplied in at every node, a deselected row carrying $\mu = 0$ and entering no per-key sum. Under $|R_i| = |R_j| = N$ this is $13N$ per edge, so the propagation costs $13(k-1)N = O(IN)$ over the tree, and the operator consuming it adds only its own read-off. Both OBJ variants hand over the same $\mathcal{R}^c$ and so face the same figure; they differ in *when* they pay it, which is what the comparison below turns on.

The aggregation of Section 6.4 is that propagation run on the clean channel, plus a read-off at the root. Per edge (R_i, R_j) it therefore costs one sort gate, $O(|R_j|)$, one group-by gate, $O(|R_j|)$, and one lookup for the multiplicity transfer, $O(|R_i| + |R_j|)$, at the committed heights above; the final aggregate is one weighted sum over the root relation. Summed over the $k-1$ edges, the total constraint count is $O(IN + OUT)$, where $IN = \sum_i |R_i|$ and OUT is the size of the final aggregated query output. Under the One-Pass OBJ this traversal is the one (9) already runs, so the aggregate adds only the read-off.

G.5 Join Results Enumeration Cost

The join results enumeration of Section 6.5 adds the following on top of whichever OBJ total applies. Occurrence binding performs one lookup per relation between the index-augmented table and the M witness references, costing $O(|R_i| + M)$ constraints per relation, i.e., $O(IN + OUT)$ over all k relations. Join-row assembly adds $(k-1)$ equality constraints per output row for key agreement plus $O(1)$ constraints per output attribute for row reproduction, and canonical ordering adds one linear packing gate per output row and $M-1$ sort-gate comparisons, all $O(OUT)$ for a fixed query. Cardinality binding reuses the multiplicity propagation at its $O(IN)$ cost (Appendix G.4), plus a single equality constraint; with the One-Pass OBJ that propagation is already produced by (9), so the binding costs only the equality. The total is therefore $O(IN + OUT)$, matching the overall OBJ bound.

G.6 One-Pass OBJ Total

The One-Pass OBJ enforces three conditions over the prover-supplied indexed partition $(\widehat{\mathcal{R}}^c, \widehat{\mathcal{R}}^r)$. Because the occurrences are index-tagged, condition (7) is certified by a Conservation Check alone, with no accompanying Non-Membership Check.

(1) *Conservation (k checks)*. One Conservation Check per relation, between $\widehat{R}_i$ and the concatenation of the two parts, at $|R_i|$ constraints each:

$$\sum_{i=1}^k |R_i| = IN.$$

Because the occurrences are index-tagged, no Non-Membership Check accompanies it, and the $8IN$ that a value-level split pays disappears, its permutation merging with the Conservation Check on the same rows. The entries of the two parts carry the constant tags 1 and 0 in the indicator position, so the same permutation also forces c to be boolean on every committed row and no separate booleanity gate is charged. The predicate gate and the certification of b apply only to relations carrying a predicate and are charged with the predicate handling of Appendix E.

(2) *Pairwise Consistency* ($2(k-1)$ checks). For every edge (R_i, R_j) , two Membership Checks verify set equality on the shared key. Each costs $\max(|R_i|, |R_j|)$ constraints, plus one gating product per row to form $c(t)(t[K_{ij}] + 1)$ on each side. The shift by one keeps 0 clear of every real key, and one reserved residual row per relation keeps 0 present in both tables, so the two mutual checks stay satisfiable when a relation happens to be entirely clean. Total:

$$O\left(\sum_{(i,j) \in T} (|R_i| + |R_j|)\right) = O(IN).$$

This step cannot be omitted: part (1) of the correctness argument rests on it, and the standalone clean-side propagation of Appendix G.4 relies on it to guarantee that every clean parent key has a matching child, so its per-key table can be built from the child keys alone; the fused pass of (3) below instead reads both channels from one merged-key table.

(3) *Cardinality Preservation (one fused counting pass)*. Per tree edge (R_i, R_j) the fused pass merges the key columns of R_i and R_j and proves the merge sorted, at $|R_i| + |R_j|$ for the permutation binding plus $8(|R_i| + |R_j|)$ for the sortedness proof. One scan over the merged rows then emits both per-key sums, at one boundary constraint and two running additions per row, i.e., $3(|R_i| + |R_j|)$. Each merged row carries a side tag that the two additions multiply in, so only child rows enter the sums; merging the parent keys is also what gives every parent row an entry to look up, with $\sigma=0$ on a key that no child row carries, and each parent's two running products are seated at its own anchor rather than at 1. Each parent tuple performs one lookup of the pair against the merged table, $|R_i| + |R_j|$, and folds the two results into its two running products, $2|R_i|$. Under $|R_i| = |R_j| = N$ this is $28N$ per edge for both counts, plus one equality per row and channel to seat each running product at the row's anchor, $2N$ per relation, which at a leaf is the whole of μ^a . The single-channel propagation of Appendix G.4 accounts for $13N$ of that, leaving $15N$ as what (9) adds on top of a pass the query already runs; the comparison below uses this split. Summed over the tree:

$$O\left(\sum_{(i,j) \in T} (|R_i| + |R_j|)\right) = O(IN).$$

Total. Summing the three components above:

$$C_{1\text{-Pass}} = IN + O(IN) + O(IN) = O(IN).$$

G.7 Comparison

Both variants achieve $O(IN)$ circuit complexity, so the advantage of the One-Pass OBJ lies in the constant factor. Under uniform relation sizes $|R_i| = N$ with $IN = kN$, each semijoin gate costs

$N + N + 8 \cdot 2N = 18N$, while a clean-side propagation costs $13N$ per edge, the fused two-channel pass $28N$, and a Conservation Check N per relation. Pairwise consistency is the same $2(k-1)$ Membership Checks in both variants, up to the gating products of the indicator realization, and cancels throughout.

Gate cost. Counting only what each variant spends to certify $\mathcal{R}^c$:

$$\begin{aligned} C_{2\text{-Pass}}^{\text{gate}} &= 36(k-1)N = (36k-36)N, \\ C_{1\text{-Pass}}^{\text{gate}} &= kN + 28(k-1)N = (29k-28)N. \end{aligned}$$

The One-Pass is the cheaper gate for every $k \geq 2$, by $(7k-8)N$: certifying a claimed reduction costs less than recomputing one, because the partition is certified by one permutation per relation and its completeness by a count. The per-edge view is the split in (3) above, where $13N$ of the fused pass is a clean-side propagation and only the remaining $15N$ is what (9) adds, against the $18N$ of the semijoin gate it replaces.

Total cost. No query stops at the gate. Aggregation reads the multiplicities at the root and the final join enumeration binds M against the root sum, so every query pays for one clean-side propagation, $13(k-1)N$. The Two-Pass pays it after the gate; the One-Pass has already produced it as one channel of its fused pass, which is the reason (9) is certified by comparing root sums. Charging it where it falls:

$$C_{2\text{-Pass}} = (49k-49)N, \quad C_{1\text{-Pass}} = (29k-28)N,$$

so the One-Pass saves $(20k-21)N$ constraints. Anchoring costs one equality per row on each channel; the clean-side half is shared with the operator, so (9) carries only the input-side half, N per relation and kN over the tree. Charging it against the margin leaves $(19k-21)N$, positive for every $k \geq 2$. The One-Pass is ahead at the gate and further ahead in total, and the $13(k-1)N$ it never spends twice is what turns the gate margin into the total one.

The saving decomposes exactly into the two structural differences. The top-down pass, $18N$ per edge or $(18k-18)N$ in total, is replaced by the k Conservation Checks at kN : over index-tagged occurrences the merge-and-sort Non-Membership Checks are unnecessary, saving $(17k-18)N$. The bottom-up pass together with the propagation, $18N + 13N = 31N$ per edge, is replaced by the single fused pass at $28N$, saving $(3k-3)N$. Most of the margin therefore comes from replacing the top-down pass with the indexed partition; the fusion contributes the smaller term. The One-Pass also materializes no intermediate reduced relations, operating directly on the prover-supplied partition.

Aggregate queries and plain joins. The two consumers of $\mathcal{R}^c$ differ in what they read but not in the margin. An aggregate reads the multiplicities at the root to form its SUM or COUNT, adding the grouped read-off. A plain join reads only the root sum, to bind M , and adds the occurrence binding, join-row assembly and canonical ordering of Appendix G.5. In both cases the added term is a function of the certified clean instance and M alone, which the two variants determine identically, so it cancels and the saving stays $(20k-21)N$. What changes is its weight. For an aggregate, OUT is the number of groups and the gate dominates the proof, so the saving is most of the difference between the variants. For a plain join, OUT is the join size and may exceed IN , leaving the same absolute saving as a smaller fraction of the whole.

Two further savings follow for the One-Pass alone: COUNT(*) is the clean-side root sum that (9) has already certified, so it costs nothing beyond the gate, and the cardinality binding of the final join enumeration reduces to a single equality constraint rather than a propagation.

H Differential Privacy Framework for Cyclic Joins

Section 7.1 introduced the Tree-Decomposed Join Gate (TDJ) and, as an *optional* relaxation applied on top of its fully oblivious construction, allowed each intra-cluster witness column to replace worst-case padding with a DP-guided upper bound on the true cardinality. This appendix provides the formal underpinning for that construction. Appendix H.1 formalizes the connection between obliviousness and differential privacy, unifying full obliviousness and its DP-based relaxation on a single (ϵ, δ) -obliviousness spectrum. Appendix H.2 then instantiates this spectrum for cyclic joins, specifying the concrete one-sided noise mechanism TDJ uses and analyzing its privacy and correctness. Appendix H.3 discusses why extending PonglyphDB with the same DP relaxation would remain substantially more expensive than VPJoin + DP.

H.1 Obliviousness and Differential Privacy

Continuing from Section 7.1, worst-case intra-cluster padding is still far from satisfactory: for queries with non-trivial treewidth, the $O(IN^w)$ padding can remain prohibitively expensive. This cost is not incidental to our realization but an inherent consequence of requiring *full obliviousness*: since the witness capacity allocated to each intermediate is part of the public circuit (see Section 3.2), it must be fixed independently of the database, otherwise a curious verifier could read off the true intermediate cardinalities directly from the published circuit. OBJ on acyclic queries satisfies this requirement essentially for free, since the circuit avoids materializing intermediate joins altogether. For cyclic queries, however, the intra-cluster intermediates introduced by tree decomposition have data-dependent cardinalities, so preserving full obliviousness forces each intra-cluster witness capacity to be padded to its worst case, $O(IN^w)$.

To achieve a more favorable intermediate size, and hence more efficient proving, a principled relaxation of full obliviousness is necessary. DP is a natural candidate precisely because it shares the same structural form as full obliviousness: both are constraints on how much a mechanism's output varies when the input database changes by a single record. Full obliviousness demands that the output be identical for any two neighboring databases, whereas DP relaxes "identical" to "probabilistically indistinguishable within a quantifiable (ϵ, δ) budget." This shared form lets us place the two notions on a single spectrum of structural leakage, which we formalize next.

DEFINITION 4 (DIFFERENTIAL PRIVACY). A randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ) -differential privacy [16] if for any two neighboring databases D_1, D_2 that differ by a single record and any set of outcomes S ,

$$\Pr[\mathcal{M}(D_1) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D_2) \in S] + \delta.$$

The privacy budgets (ϵ, δ) control the level of the privacy guarantee, where smaller values correspond to stronger privacy protection.

Embedding full obliviousness into a DP framework. Obliviousness itself is a deterministic notion: it is defined purely by requiring identical circuits across same-parameter databases. However, we can

embed it into a DP framework so that full obliviousness and a DP-based relaxation sit on a single spectrum. View the circuit generator $\mathcal{G}(\cdot)$ as a mechanism from databases to circuits. It can be shown that any $\mathcal{G}(\cdot)$ satisfying full obliviousness is $(0, 0)$ -DP: since $\mathcal{G}(\cdot)$ produces the same output on any two databases sharing the same public parameters, we have $\Pr[\mathcal{G}(D_1) \in S] = \Pr[\mathcal{G}(D_2) \in S]$ for every neighboring pair D_1, D_2 and every set S . As a result, we propose to unify full obliviousness and its DP-based relaxation under a single notion, which we call (ϵ, δ) -obliviousness: a circuit generator $\mathcal{G}(\cdot)$ is (ϵ, δ) -oblivious whenever it satisfies (ϵ, δ) -DP as a mechanism from databases to circuits. This yields a unified spectrum of structural leakage parameterized by (ϵ, δ) :

- $\epsilon = 0, \delta = 0$ (deterministic): full obliviousness, where circuit dimensions are fully data-independent (with worst-case padding).
- $0 < \epsilon < \infty, \delta$ negligible: *DP-guided relaxation*, where circuit capacities satisfy (ϵ, δ) -DP.
- $\epsilon \rightarrow \infty$: *no privacy*, where capacities equal the true cardinalities.

We emphasize that (ϵ, δ) parameterize the *relaxation*, not obliviousness itself: full obliviousness remains a deterministic equality-based notion. Within this framework, ZK protects the content of witness columns, while DP relaxes full obliviousness on the observable circuit dimensions and still ensures formal privacy. We instantiate the middle regime next.

H.2 DP-Guided Relaxation for Cyclic Joins

With the DP-guided relaxation of full obliviousness (Appendix H.1), we are now able to set each intra-cluster circuit capacity to a data-dependent upper bound on the true cardinality, rather than its worst-case bound $O(IN^w)$. The capacity must *upper-bound* the true size, since underestimating would discard valid tuples and break correctness. To guarantee this upper-bound property while still providing formal privacy, we adopt the one-sided noise mechanism of Wang et al. [45], which yields (ϵ, δ) -DP on the released capacity and ensures privacy of the intermediate cardinalities.

Incentive-aligned privacy model. A key observation is that in our ZK database setting, the DP noise protects the *prover's own data* from the verifier. This creates a natural alignment of incentives: any manipulation of the DP mechanism—using too little noise, too much noise, or incorrect sensitivity estimates—harms only the prover, either through privacy loss or increased computation. The verifier is unaffected, since it only cares about the correctness of the final query result, not the intermediate sizes. This incentive alignment allows us to omit in-circuit verification of DP parameters entirely: the circuit need not prove that noise was sampled correctly or that ϵ meets any threshold. These are the prover's concern alone.

DP upper bound realization. To obtain the DP upper bound on the cardinality of the in-cluster join result m , we adopt the *one-sided noise mechanism* [45], which guarantees that the introduced DP noise is non-negative. The noise scale is computed from the maximum frequency of each relation, following the analysis in [14]. To preserve privacy during the computation of the noisy scale, these maximum frequencies must themselves be made DP-compliant, again by applying the one-sided noise mechanism. We denote the resulting DP upper bound on m by $\tilde{m}$. By construction, $\tilde{m} \geq m$ while preserving (ϵ, δ) -DP.

Circuit realization. The circuit implementation is straightforward. The prover sets each intra-cluster witness column capacity to the noisy bound. The circuit processes all rows uniformly, using standard “is_dummy” selector columns to exclude padding from downstream computations. No additional constraints are needed for the DP mechanism itself. The sampling and calibration are therefore an offline prover-side routine rather than part of the circuit. That routine ships with the artifact and its random seed is fixed, so the released capacities can be regenerated and checked against the ones the reported circuits use.

Correctness. The correctness of TDJ follows from two independent arguments. First, the inter-cluster join is verified by the One-Pass OBJ (Section 6.3), whose correctness is established in Appendix F.2. Second, each intra-cluster join is verified using binary join gates [26] that enforce key matching via lookup arguments. The DP mechanism affects only the *capacity* (padding size) of the intra-cluster witness columns, not the correctness constraints themselves: the one-sided noise guarantees $\tilde{m} \geq m$, so all valid tuples fit within the allocated capacity and no results are lost.

H.3 Comparison with a Hypothetical PoneglyphDB + DP Extension

R2.O1 Since DP-guided padding relaxes full obliviousness, a natural question is whether the baseline could benefit equally: one could imagine extending PoneglyphDB with the same relaxation, replacing each worst-case-padded intermediate of its binary-join chain with a DP upper bound on the intermediate’s cardinality. Such a hypothetical PoneglyphDB + DP extension would remain substantially slower than VPJoin + DP for cyclic joins, for two reasons rooted in how the two systems structure the join.

First, the noise magnitude scales with the join sensitivity. PoneglyphDB must decompose the whole k -way join into $k-1$ binary joins and release a DP-protected capacity after each of them. Because a single input tuple can join with tuples in every downstream relation, the sensitivity of a deep intermediate grows exponentially with the number of joins below it, so PoneglyphDB’s noise must be calibrated to the sensitivity of (up to) the full k -way join. VPJoin, in contrast, confines materialization to the small clusters of the decomposition tree: inside a cluster it uses the same binary-join gates as PoneglyphDB, but the inter-cluster joins are verified by OBJ without materializing any intermediate. Consequently, VPJoin only adds noise proportional to the sensitivity of a *partial* join, i.e., the join inside a single cluster, which is far smaller.

Second, the privacy budget must stretch much further in PoneglyphDB + DP. Because PoneglyphDB forms a chain of binary joins, a tuple in one relation is involved in all $k-1$ of them, so the overall (ϵ, δ) budget must be divided across all $k-1$ releases. A larger sensitivity or a smaller per-step privacy budget directly increases the DP noise added to each intermediate capacity; since the circuit must be padded to this noisy upper bound, more noise means more dummy rows, a larger circuit, and therefore a longer proving time. VPJoin materializes only a small constant number of clusters, so its budget is split only a few ways. Together, these two effects imply that the gap between the two systems grows with the join depth k even when both are granted the same DP relaxation.

I Queries

The complete 4 graph queries are shown below:

I.1 Graph Queries

GQ 1: 3-way path.

```
SELECT COUNT(*) AS cnt
FROM Edge r1
JOIN Edge r2 ON r1.dst = r2.src
JOIN Edge r3 ON r2.dst = r3.src
WHERE r1.src < r2.src AND r2.src < r3.src;
```

GQ 2: 4-way path.

```
SELECT COUNT(*) AS cnt
FROM Edge r1
JOIN Edge r2 ON r1.dst = r2.src
JOIN Edge r3 ON r2.dst = r3.src
JOIN Edge r4 ON r3.dst = r4.src
WHERE r1.src < r2.src
AND r2.src < r3.src
AND r3.src < r4.src;
```

GQ 3: 3-way cycle.

```
SELECT COUNT(*) AS cnt
FROM Edge r1
JOIN Edge r2 ON r1.dst = r2.src
JOIN Edge r3 ON r2.dst = r3.src
AND r3.dst = r1.src
WHERE r1.src < r2.src AND r2.src < r3.src;
```

GQ 4: 4-way cycle.

```
SELECT COUNT(*) AS cnt
FROM Edge r1
JOIN Edge r2 ON r1.dst = r2.src
JOIN Edge r3 ON r2.dst = r3.src
JOIN Edge r4 ON r3.dst = r4.src
AND r4.dst = r1.src
WHERE r1.src < r2.src
AND r2.src < r3.src
AND r3.src < r4.src;
```

I.2 Cyclic Query Decomposition

We describe how the three cyclic queries (Q5, GQ3, GQ4) are decomposed into acyclic cluster trees for the TDJ gate (Section 7.1).

TPC-H Q5. We abbreviate the six tables as C (customer), O (orders), L (lineitem), S (supplier), N (nation), R (region), and write ok , ck , sk , nk , rk for orderkey, custkey, supkey, nationkey, regionkey. Q5 is a 6-way join whose join graph contains a 4-way cycle among C , O , L , and S : they are connected through both the transactional path ($O.ck=C.ck, L.ok=O.ok, L.sk=S.sk$) and the shared nationality constraint ($C.nk=S.nk$). The remaining tables N and R

join acyclically via nk and rk and thus do not require materialization into bags. We break the cycle by grouping the four cyclic relations into two bags:

- **Bag 1** $\{O, C\}$: materialize $O \bowtie C$ with the date filter, producing tuples (ok, nk) .
- **Bag 2** $\{L, S\}$: materialize $L \bowtie S$, producing tuples $(ok, nk, price, disc)$.

The inter-cluster join tree has four nodes—Bag 1, Bag 2, N , R : Bag 2 joins Bag 1 on (ok, nk) , Bag 2 joins N on nk , and N joins R on rk . This tree is acyclic, so the One-Pass OBJ verifies all inter-cluster joins. Only the two bags require intra-cluster materialization; their capacities are set to their worst-case bounds under full obliviousness or, in the optional VPJoin + DP mode, to DP-guided noisy bounds (Section H.2).

GQ3 (3-way cycle / triangle). GQ3 counts triangles $A \rightarrow B \rightarrow C \rightarrow A$ with $A < B < C$ over the $\text{Edge}(src, dst)$ relation, using three self-join instances r_1, r_2, r_3 . The cycle arises from the closing edge $r_3.dst = r_1.src$. We decompose via the separator (A, C) :

- **Bag 1** $\{r_1, r_2\}$: materialize the 2-hop path $A \rightarrow B \rightarrow C$ (wedges).
- **Bag 2** $\{r_3\}$: the closing edge $C \rightarrow A$.

The separator variables are (A, C) . Bag 2 sends a message to Bag 1: for each distinct (A, C) key, the count of closing edges $C \rightarrow A$. The final answer aggregates over Bag 1, weighting each wedge by the Bag 2 count and enforcing $A < B < C$.

GQ4 (4-way cycle). GQ4 counts 4-cycles $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ with $A < B < C < D$ over the $\text{Edge}(src, dst)$ relation, using four self-join instances r_1, r_2, r_3, r_4 . The cycle arises from $r_4.dst = r_1.src$. We decompose via a diagonal cut on (A, C) :

- **Bag 1** $\{r_1, r_2\}$: materialize the path $A \rightarrow B \rightarrow C$.
- **Bag 2** $\{r_3, r_4\}$: materialize the path $C \rightarrow D \rightarrow A$.

The separator variables are (A, C) . Bag 2 sends a message to Bag 1: for each (A, C) key, the count of paths $C \rightarrow D \rightarrow A$ satisfying $C < D$. The final answer aggregates over Bag 1, weighting each path $A \rightarrow B \rightarrow C$ by the Bag 2 count and enforcing $A < B < C$.

J Estimation Details for PoneglyphDB

This appendix details the estimation methodology of Section 8.1: how the padded gate-row count N is derived, how the anchors are implemented and measured, and why the resulting estimate is conservative.

Padded gate rows. PoneglyphDB evaluates a k -way join $R_1 \bowtie \dots \bowtie R_k$ as a chain of binary joins that materializes every intermediate: $P_2 = R_1 \bowtie R_2$ and $P_t = P_{t-1} \bowtie R_t$. Each level occupies its own columns of one circuit, so the gate-row count N is the height of the tallest level, dominated by the last intermediate's capacity. Once no schema constraint is assumed (Section 8.1), the only capacity the public information implies for P_t is the product $\prod_{s \leq t} |R_s|$ of the public cardinalities of the relations already joined. These cardinalities are used as they actually are, one per relation, and are not replaced by a common bound: a query over small dimension tables is therefore charged far less than its relation count alone would suggest. Tighter capacities would require the schema assumptions our evaluation excludes; the qualitative conclusion is insensitive to this choice, since the padded circuits remain far beyond executable size either way.

Writing Π_Q for the capacity that full obliviousness forces on that tallest level, i.e. the product of the public sizes of the relations combined into it, the padded domain is $G_C = 2^{\lceil \log_2 \Pi_Q \rceil}$, the smallest power of two holding that level, and the estimate charges the anchor's measured per-domain-row rate for every row of it: $T_{\text{est}} = T_0 \cdot G_C / G_{C,0}$, with $G_{C,0}$ the anchor's own domain. Only the anchor's measured time, its domain, and the public relation sizes enter, so every reported PoneglyphDB estimate can be recomputed from Table 1 and the sizes stated below.

Which relations are combined into the tallest level is fixed by the plan each query is evaluated under, and both systems are charged the same plan. The tallest level is the final one, P_k : the chain materializes the join output before the aggregation operator consumes it, and only the aggregate is revealed, so under full obliviousness the output level is padded to its capacity like every other. For the scalar graph counts that level's true cardinality happens to equal the revealed answer, so a baseline that sized this one level by the disclosed result would fall back to the previous level's capacity; we charge the plan as published, with result-independent capacities, which is also the convention PoneglyphDB's own reported estimates follow. For TPC-H at the 60K scale the fact table `lineitem` (60,175 rows) is joined last; every level below the final one then excludes it, so this is the cheapest plan for the baseline, while the final level's capacity is the same under every order. The full products give $\Pi_Q = 1.35 \times 10^{12}$ for Q3 and Q18, 1.69×10^{16} for Q5, 3.61×10^{19} for Q9 and 2.71×10^{23} for Q8, i.e. exponents $\lceil \log_2 \Pi_Q \rceil$ of 41, 54, 65 and 78. In the graph queries every relation is a copy of `Edge` with m rows, so a k -way query gives $\Pi_Q = m^k$. The 3-way queries GQ1 and GQ3 therefore give m^3 , an exponent of 45 on LastFM and 50 on Facebook and Wikipedia, while the 4-way queries GQ2 and GQ4 give m^4 , exponents of 60, 66 and 67. Arity raises the padding factor because the baseline feeds each binary join into the next, so one more relation multiplies the tallest level by another m ; how VPJoin avoids that compounding is described next.

Anchors. The TPC-H anchors run the released PoneglyphDB artifact, unpadded, on the same server and build profile as every other measurement. Their domain is $G_{C,0} = 2^{16}$, the degree every TPC-H circuit uses at the 60K scale, where `lineitem` holds 60,175 rows. For the graph queries that artifact provides no circuits, and we do not reimplement PoneglyphDB's execution strategy. The anchor is instead VPJoin's own circuit for the query, run with the true join and intermediate sizes revealed in place of the padded capacities that full obliviousness requires; this is the same construction the *Revealing Join Size* configuration uses, so it is a real proof of the query as specified rather than a cost stub. What the anchor supplies is therefore a measured wall-clock time for a PLONKish circuit of known domain size on the target server and build profile, and the estimate scales that time to the padded PoneglyphDB domain. The step assumes that per-row proving cost is comparable between the two circuits; it is not an assumption that they share a gate structure, which for the graph queries they do not. Each graph anchor is one dense run per query, serving all three datasets, since the constraint system depends only on the query shape and the dataset only sets the row count: GQ1 and GQ2 prove the first 65,520 edges of Wikipedia in a 2^{16} domain (99.98% full), and GQ3 and GQ4 prove the first 21,403 edges of LastFM, whose true wedge of 131,001 rows fills a 2^{17} domain to 99.95%, with capacities set to the true sizes. Filling the domain is what makes the rate transferable:

Table 1: Anchor executions underlying the estimates. Left: measured proving time of the PoneglyphDB artifact (TPC-H at the 60K scale). Right: measured wall-clock time of VPJoin’s dense graph anchors, including key generation; one anchor per query serves all three datasets.

Query	T_0 (s)	$G_{C,0}$	Query	T_0 (s)	$G_{C,0}$
Q3	111	2^{16}	GQ1	70	2^{16}
Q5	127	2^{16}	GQ2	98	2^{16}
Q8	168	2^{16}	GQ3	125	2^{17}
Q9	157	2^{16}	GQ4	164	2^{17}
Q18	105	2^{16}			

rows the layout never assigns are skipped by the commitment MSM, so a sparse anchor under-measures the per-row rate a fully padded circuit pays. T_0 is the anchor’s wall-clock time including key generation, whose commitment work also tracks the assigned rows and therefore scales with a padded domain. On LastFM the closing edge of the cyclic patterns never occurs, so the GQ3 and GQ4 anchors prove the count 0; the wedge, every lookup and every permutation still carry real data on all rows, and the two all-zero accumulator columns bound the distortion to a few percent. The per-domain-row rates are 1.07, 1.49, 0.95 and 1.25 ms for GQ1–GQ4; for comparison, the artifact’s own TPC-H anchors run at 1.6 to 2.6 ms per domain row, so transferring VPJoin’s rate to the padded baseline under-charges it. Every anchor proof is verified, and $G_{C,0}$, T_0 , both capacity bounds, and the resulting estimates are recorded for every anchor. Table 1 reports the measured wall-clock time of every anchor. Each estimate is the anchor’s per-domain-row rate times the padded domain, $T_0 \cdot G_C / G_{C,0}$, as derived above. The same rate construction, from the same anchors, yields the fully oblivious VPJoin times for GQ3 and GQ4, with G_C set by TDJ’s worst-case intra-cluster capacities over the decompositions of Appendix I.2 in place of PoneglyphDB’s padded products, derived below.

Padded domains for the fully oblivious VPJoin estimates. The same two steps give VPJoin’s own fully oblivious times for the cyclic graph queries, with TDJ’s capacities in place of PoneglyphDB’s padded products. TDJ pads only the intra-cluster materializations, never the inter-cluster joins, which OBJ verifies without materializing them: GQ3 is evaluated over a single bag and GQ4 over two bags sharing one materialization read in two roles, so a bag is bounded on its own instead of feeding the next binary join, and the capacity does not grow with arity the way the baseline’s does. Each bag is the wedge of two *Edge* relations joined on the middle vertex, and over all m -edge graphs that wedge is largest for a star, at $\lfloor m/2 \rfloor \cdot \lceil m/2 \rceil$, so the capacity is a function of the public edge count alone and rests on exactly the information the baseline’s bounds use; a declared public degree bound would tighten it, but we assume none, matching the no-schema-constraint setting. The resulting proving domains are $G_C = 2^{28}$ on LastFM, 2^{31} on Facebook and 2^{32} on Wikipedia, and $T_{\text{est}} = T_0 \cdot G_C / G_{C,0}$ from the same per-query anchors of Table 1 gives the times reported in Section 8.2. The acyclic GQ1 and GQ2 pad nothing at all, so their circuits hold the committed edge rows and their times are measured rather than scaled.

Conservativeness of the estimate. Padding does not change what a row costs the prover: polynomial commitments, lookup grand products, and quotient evaluations process every domain row identically whether it holds a live tuple or a padding dummy, which is exactly what obliviousness requires. The anchor’s per-domain-row rate therefore transfers to the padded circuit unchanged, giving $T_{\text{est}} = T_0 \cdot G_C / G_{C,0}$. An obliviousness dummy carries field values like any live tuple, which is what distinguishes it from a row the layout never assigns; the latter is identically zero and is skipped by the commitment MSM.

The two asymptotic deviations from linearity run in opposite directions and are of the same logarithmic order: the $O(N \log N)$ FFT factor raises the cost, while the MSM term lowers it, since Pippenger’s window widens with the input length and the cost per point falls with it. At directly measurable ratios they cancel: the same GQ3 anchor circuit proved over a $32\times$ larger domain, 2^{17} to 2^{22} , costs a measured $27.6\times$ in proving time. Over the far larger ratios the estimates use, the FFT factor accumulates and the omission reaches a factor of roughly 1.5 to 2.5 at the largest exponents. Key generation sits inside the graph anchors’ T_0 , and its commitment work tracks the assigned rows, so on a fully padded domain it scales like the rest; the TPC-H artifact anchors report proving time only, which understates the baseline there further. What the linear model omits unambiguously in PoneglyphDB’s favor is memory, since the padded circuits’ polynomials exceed the server’s capacity by orders of magnitude, so a faithful execution would additionally pay paging or distribution costs. The reported estimates therefore under-state the cost of the padded circuits.

K Cost of Binding Proofs to the Database Commitment

Our implementation inherits a further simplification of the PoneglyphDB harness (Section 8.1): it benchmarks the query-verification circuits with a trivial public input and does not bind the published *Commit(D)* (Appendix A) to each proof, since Halo2’s internal commitments already provide within-proof binding. We therefore implement this binding inside each query circuit: the check is inlined and copy-constrained to the circuit’s own witness cells, so a query and its binding form one proof, and its extra cost is the difference between that proof and the plain one, plus the column openings. At Setup, and before any query is issued, every attribute column the workload reads is committed separately under its own random blinder, over the smallest evaluation domain that covers the database ($k = 16$ for TPC-H, the same domain the query circuits already use; 2^{15} for the LastFM edge columns and 2^{17} for Facebook and Wikipedia). The layer thus reuses a prefix of the circuits’ existing parameters and needs no separate setup. That publication is *Commit(D)*, and it is made once: the points and the blinders are fixed there, and a query opens the columns it reads out of them rather than committing again. This is what makes the binding hold *across* queries, since two queries reading a column open the same published commitment; committing per query under a fresh blinder would leave each proof bound only to a commitment created for it, and nothing would tie two answers to one database.

Binding a proof to *Commit(D)* is a random-point consistency check, applied to exactly the columns that query’s circuit witnesses (8–19 columns for TPC-H; the two *Edge* columns for the graph queries), in that circuit’s own degree. A challenge x is derived by Fiat–Shamir from the whole publication, the indices of the columns the query

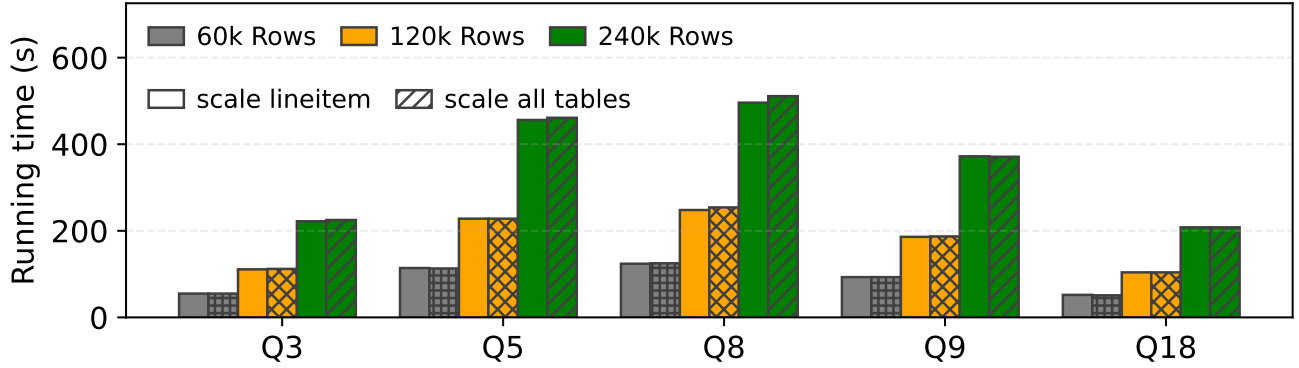


Figure 9: Scaling only `lineitem` (solid) against scaling every table proportionally (hatched), at equal `lineitem` size. The two bars of each pair coincide, so the dimension tables add no measurable cost.

opens, and the identity of the query; the prover opens each of those columns at x , while the circuit evaluates its own witness column at x by a Horner accumulation and exposes the value as a public output, copy-constrained cell by cell to the column the query’s own gates read. Absorbing the whole publication, rather than only the columns read, is what prevents a challenge from being carried over to a second publication of the database. In our implementation the query and its binding form one proof, so there is no separate query proof for x to absorb: x is fixed by the publication, which is binding before any challenge exists, and the openings tie the checked values to it. What the random-point argument rests on is a witness fixed before x , since a challenge known in advance is one against which a prover may choose a witness $W \neq D$ with $W(x) = D(x)$; Halo2 fixes a proof’s public inputs before its witness commitments, so a single proof cannot derive x after committing its own witness, and the layer therefore assumes the witness columns are fixed with the publication. This is the setting of the threat model, a prover that commits its database once at Setup and answers queries against it. Under that assumption, two distinct columns agree at x with probability at most $\deg/|\mathbb{F}|$, so matching values certify that the witness columns equal the committed data, which is the in-circuit consistency step that discharges Threat (i). The extra cost per proof is therefore the in-circuit check, the column openings, and their verification, on top of the one-time Setup that publishes $\text{Commit}(D)$. That publication is 32 bytes per column of D ; for TPC-H the five queries witness 69 columns between them but only 34 distinct ones, so the publication is 1088 bytes and costs a measured 0.159 s. The 35 shared readings are exactly where the cross-query binding bites: each is one published point that every query reading it opens.

Table 2 reports the total extra cost per proof: 5.9–16.7 s for the TPC-H queries and 0.9–86.5 s for the graph queries, which is 11–15% of the corresponding proving times for TPC-H and 2–4% for the graph queries. The openings and their verification are measured on every row; the in-circuit component is measured as the difference between the combined and the plain proof where that difference resolves above the machine’s run-to-run variation. The variation across the TPC-H queries follows the number of bound columns, from 8 for Q18 to 19 for Q8, with Q5 largest because its DP-padded circuit proves over a doubled domain. The graph queries all bind the same two Edge columns, so the cost is set by the proving domain rather than the query shape: GQ1 and GQ2 agree on every dataset, and GQ3 and GQ4 pay proportionally more over their DP-padded domains. Across datasets

Table 2: Total extra cost per proof (seconds) of binding the query circuits to the published column commitments $\text{Commit}(D)$ (TPC-H at the 60K scale): the in-circuit check plus the column openings.

Query	Extra (s)	Query	LastFM	Facebook	Wiki
Q3	7.56	GQ1	0.94	3.74	3.80
Q5	16.73	GQ2	0.95	3.68	3.80
Q8	13.79	GQ3	10.72	75.88	75.82
Q9	10.91	GQ4	10.92	86.50	83.76
Q18	5.86				

the cost tracks the domain the edge columns force, which is also what sizes the openings.

L Scaling All Tables Proportionally

Figure 4 scales only `lineitem`, leaving the dimension tables at their base sizes. To isolate the effect of the dimension tables, we generated a second family of datasets in which *every* table grows with the scale, and compared the two families back to back.

Datasets. Both families are produced by the official TPC-H dbgen toolkit. The all-tables family uses scale factors 0.01, 0.02 and 0.04, so `orders`, `customer`, `part`, `partsupp` and `supplier` grow by the same 2× and 4× factors as `lineitem` (60,175, 120,515 and 240,292 rows); `nation` and `region` are the fixed TPC-H catalogs and stay at 25 and 5 rows. The `lineitem`-only family holds every dimension table at the base scale factor and grows `lineitem` alone to the same three sizes. Both families satisfy referential integrity at every scale, so no join loses tuples to dangling keys.

Result. Scaling the dimension tables leaves the cost essentially unchanged. Table 3 reports the measured ratio between the two families: it lies in $[0.99, 1.03]$ across all five queries and all three scales, even though the all-tables datasets carry up to 79,800 additional dimension rows at the 240K scale. Several of the ratios fall at or below 1 in the unrounded measurements, so what remains is run-to-run variation rather than a systematic cost. Figure 9 plots the two families together: the bars of each pair coincide.

This does not contradict the $O(IN + OUT)$ cost of Appendix G.6. The per-table checks, semijoin and membership, sortedness and range,

Table 3: Measured ratio of proving time when all tables are scaled to proving time when only `lineitem` is scaled, at equal `lineitem` size. A ratio of 1 means the dimension tables cost nothing.

Query	60K	120K	240K
Q3	1.00	1.01	1.01
Q5	1.00	1.00	1.01
Q8	1.01	1.02	1.03
Q9	1.00	1.01	1.00
Q18	0.99	1.00	1.00

and the tuple-encoding lookups, are each linear in their own table’s row count, so the constraint count does grow when every table is enlarged. What the prover is charged for, however, is not the number of rows a table occupies but the padded power-of-two domain over which Halo2 commits to and evaluates every column, since proving time is near-linear in the domain size (Appendix A). VPJoin assigns each relation its own group of advice columns, so the height of that domain is the largest single section rather than the sum of the inputs. At every TPC-H scale factor the largest section is `lineitem` itself, which is why the degree is identical in the two families, $k = 16, 17$ and 18 at the three scales. The additional dimension rows land in rows that `lineitem` has already forced the circuit to allocate, and so cost nothing.

The same model accounts for the growth *across* scales, where each doubling of `lineitem` doubles the domain and hence the time, and it also delimits the claim. The dimension tables are free only while they remain below the fact table. If a dimension table grew to rival `lineitem`, or if proportional scaling pushed the largest section past the next power of two, the domain would double and the cost would step up with it; the $k = 16 \rightarrow 17 \rightarrow 18$ progression measured here is exactly such a sequence of steps, driven by `lineitem`.

M Detailed

Privacy Budget Sensitivity for Graph Queries

This section provides a detailed, per-dataset analysis of the privacy budget sensitivity for the cyclic graph queries GQ3 and GQ4, complementing the summary presented in Section 8.4. For each query, Figures 10 and 11 plot the proving time of three configurations—Revealing Join Size (blue, dashed), VPJoin + DP (orange, solid), and VPJoin with full obliviousness (green, dashed)—as a function of ϵ on the LastFM, Facebook, and Wikipedia Vote datasets. Note that the y-axes use a broken scale: the upper portion shows the (estimated) VPJoin proving time and the lower portion shows the VPJoin + DP and Revealing Join Size proving times.

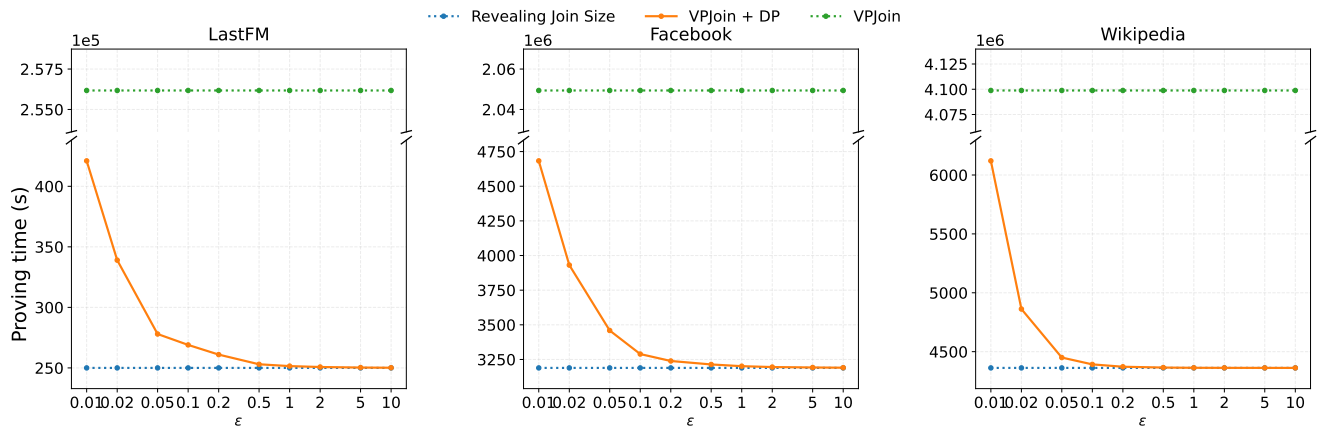
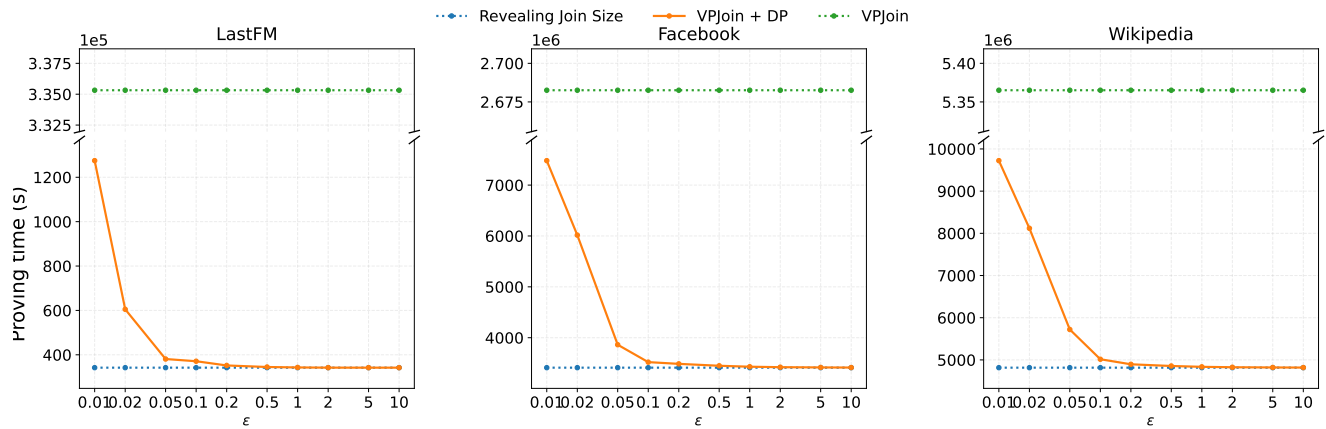
GQ3 (3-way cycle). Figure 10 reports the ϵ -sensitivity for GQ3. On LastFM (27,806 edges), the VPJoin + DP proving time decreases from approximately 421 s at $\epsilon = 0.01$ to 250 s at $\epsilon \geq 2$, converging to the Revealing Join Size baseline of roughly 250 s. The fully oblivious VPJoin time is estimated at approximately 2.6×10^5 s, which is entirely infeasible in practice. On Facebook (88,234 edges), the DP curve ranges from approximately 4,680 s ($\epsilon = 0.01$) down to 3,190 s ($\epsilon \geq 2$), against a Revealing Join Size baseline of roughly 3,190 s and a VPJoin

estimate of 2.0×10^6 s. On Wikipedia (103,689 edges), the DP curve ranges from approximately 6,120 s ($\epsilon = 0.01$) to 4,360 s ($\epsilon \geq 2$), with the baseline at roughly 4,360 s and the VPJoin estimate at 4.1×10^6 s.

Across all three datasets, the DP overhead at the default $\epsilon = 0.1$ is modest: roughly 8% above the Revealing Join Size baseline on LastFM, 3% on Facebook, and 1% on Wikipedia. This overhead arises from the additional padding rows introduced by the Laplace noise, which translate directly into extra circuit constraints. As ϵ increases beyond 1, the noise magnitude becomes negligible relative to the true intermediate cardinalities, and the DP curve converges to the no-privacy baseline.

GQ4 (4-way cycle). Figure 11 reports the analogous results for GQ4. On LastFM, the DP curve decreases from approximately 1,275 s ($\epsilon = 0.01$) to 342 s ($\epsilon \geq 2$), converging to the Revealing Join Size baseline of roughly 342 s. The VPJoin estimate is approximately 3.4×10^5 s. On Facebook, the range is approximately 7,480 s ($\epsilon = 0.01$) to 3,410 s ($\epsilon \geq 2$), with a baseline of roughly 3,410 s and a VPJoin estimate of 2.7×10^6 s. On Wikipedia, the DP curve ranges from approximately 9,720 s ($\epsilon = 0.01$) to 4,820 s ($\epsilon \geq 2$), against a baseline of roughly 4,820 s and a VPJoin estimate of 5.4×10^6 s.

Compared to GQ3, the absolute proving times for GQ4 are higher across all datasets, reflecting the additional cost of decomposing and padding a 4-way cycle versus a 3-way cycle. However, the relative DP overhead remains small: at $\epsilon = 0.1$, the overhead is approximately 9% on LastFM, 3% on Facebook, and 4% on Wikipedia. The convergence behavior is consistent with GQ3: moderate privacy budgets ($\epsilon \geq 0.5$) bring the DP-guided proving time within a few percent of the no-privacy lower bound, while the fully oblivious alternative remains about three orders of magnitude more expensive.

Figure 10: Proving time of graph query GQ3 for various ϵ values on 3 network datasets.Figure 11: Proving time of graph query GQ4 for various ϵ values on 3 network datasets.